14 haftalık yol haritasında 5. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 05

Temel Sıralama Algoritmaları

Seçmeli, Kabarcık ve Araya Ekleme Sıralaması

MODÜL II · ARAMA VE SIRALAMA · H4–H7

HAFTANIN BÜYÜK SORUSU

Elinizdeki iskambil kâğıtlarını sıralarken hangi algoritmayı kullanıyorsunuz ve bir milyon tamsayıyı sıralamak için kabarcık sıralaması neden "yanlış yol"dur?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları.....4
1.1 Öğrenme çıktıları.....5
2. Geçen Haftadan ve AP-1'den Bağlantı.....5
3. Kavramsal Anlatım: Sıralama Problemi ve Algoritma Özellikleri.....6
3.1 Sıralama problemi ve neden sıraladığımız.....6
3.2 Neyi sayıyoruz? Karşılaştırma, takas ve kaydırma.....6
3.3 Ters çevrim: bir dizinin “ne kadar sırasız” olduğu.....6
3.4 Algoritma özellikleri: kararlı, yerinde, uyarlanabilir, çevrimiçi.....8
3.5 Karşılaştırıcı sözleşmesi ve genel sıralama arayüzü.....9
3.6 Çok anahtarlı sıralama ve Türkçe alfabe.....11
3.7 Dolaylı sıralama: kayıtları değil işaretçileri taşımak.....13
3.8 Kaputun Altında: bellek trafiği, memmove ve melez kütüphaneler.....14
4. Algoritmalar.....15
4.1 Seçmeli sıralama (selection sort).....15
4.2 Kabarcık sıralaması (bubble sort).....17
4.3 Araya ekleme sıralaması (insertion sort).....21
4.4 Merak: Shell sıralaması (Shellsort).....24
4.5 Üç algoritmanın karşılaştırması.....25
5. Çözümlü Örnekler.....27
5.1 Örnek 6: “sıralı mı?” ve kâhin testi.....28
5.2 Örnek 8: yapı dizisini karşılaştırıcıyla sıralamak.....29
5.3 Örnek 11: k-sıralı dizide araya ekleme, O(nk).....30
6. Haftanın Deneyi: Algoritma × Girdi Türü.....31
6.1 Sayaçlar: ısı haritası.....31
6.2 Süreler ve büyüme.....32
6.3 Büyük Sorunun sayısal yanıtı: bir milyon tamsayı.....33
6.4 Dil sabitleri ve derleme düzeyi.....34
6.5 Sınırlılıklar ve açık kalan soru.....34
7. Hata Avcısı.....34
8. Dil Penceresi: C · C++ · Python.....37
9. Mühendis Gözüyle: Kütüphaneyi Kullan, Ama Güvencesini Oku.....38
10. Mini-OBS 2 Adımı: v2.4 — Çok Ölçütlü Sıralı Listeler.....38
10.1 Gereksinimler.....38
10.2 İpuçları.....39

10.3 Kabul ölçütleri 39
10.4 Test senaryoları 40
11. Alıştırmalar ve Atölyeler 40
11.1 İzleme ve çıktı tahmini (A) 40
11.2 Karmaşıklık (B) 41
11.3 Değişmez yazma (C) 42
11.4 Hata bulma (D) 42
11.5 Parsons (E) 43
11.6 Kod yazma (F) 43
11.7 Diller arası okuma (G) 44
11.8 Atölye 1 · İnsan sıralama makinesi (gruplar, ≈ 20 dk) 45
11.9 Atölye 2 · Sıralama laboratuvarı (eşli, ≈ 25 dk) 45
11.10 Atölye 3 · Karşılaştırıcı atölyesi (eşli, ≈ 25 dk) 45
12. Kendini Değerlendirme 46
13. İleri Bakış ve Merak Kutusu 46
14. Özet, Kavram Haritası ve Terim Sözlüğü 47
15. Kaynaklar 48
Ek A. Dil Karşılıkları 49
A.1 Örnek 1 · Seçmeli sıralama 49
A.2 Örnek 2 · Kabarcık sıralaması (erken çıkışlı) 50
A.3 Örnek 3 · Araya ekleme sıralaması 51
A.4 Örnek 4 · Üç algoritma, üç girdi: sayaçlar 52
A.5 Örnek 5 · Ters çevrim ve kaydırma 52
A.6 Örnek 6 · “Sıralı mı?” ve kâhin testi 53
A.7 Örnek 7 · qsort ve taşmaya güvenli karşılaştırıcı 54
A.8 Örnek 8 · Yapı dizisini sıralama 55
A.9 Örnek 9 · Kararlılık deneyi 56
A.10 Örnek 10 · Çok anahtarlı sıralama 57
A.11 Örnek 11 · k-sıralı dizide araya ekleme 57
A.12 Örnek 12 · Dolaylı sıralama 58
A.13 Örnek 13 · Shell sıralaması (merak) 59

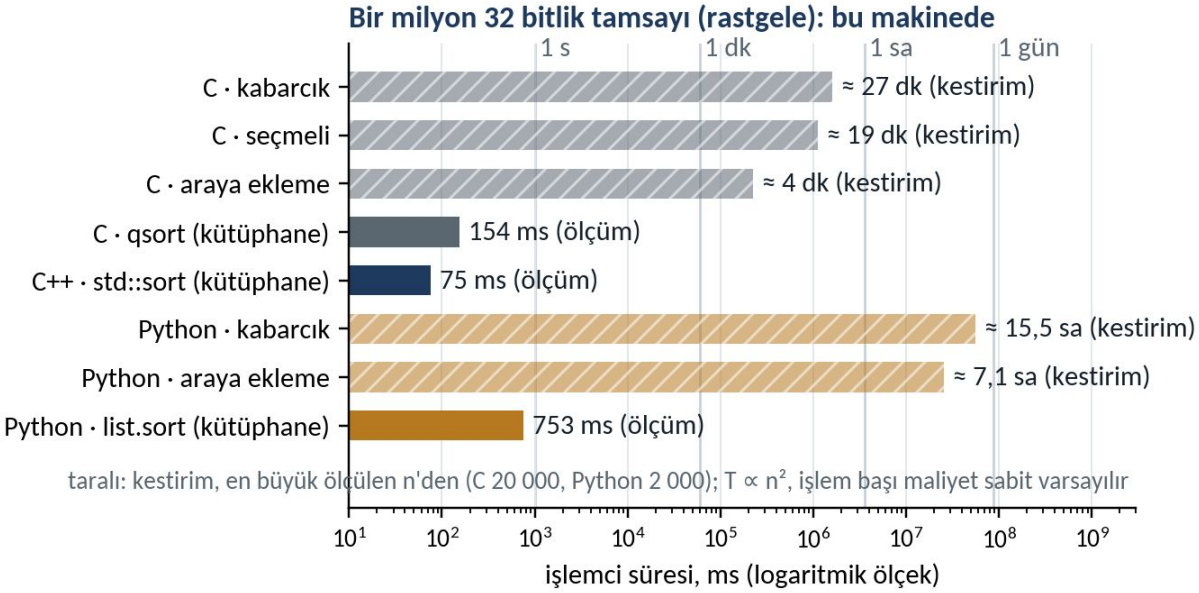
1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Elinizdeki iskambil kâğıtlarını sıralarken hangi algoritmayı kullanıyorsunuz ve bir milyon tamsayıyı sıralamak için kabarcık sıralaması neden "yanlış yol"dur?

Elinize karışık bir deste iskambil kâğıdı aldığınızı düşünün. Çoğumuz kâğıtları tek tek çeker, solumuzda sıralı tuttuğumuz elin içinde doğru yere kaydırarak yerleştiririz. Bu, bu haftanın üçüncü algoritmasının, **araya ekleme sıralamasının** (insertion sort) ta kendisidir. On üç kâğıt için mükemmel çalışır. Peki aynı yöntem bir milyon sayıda da işe yarar mı? Ya "komşuları karşılaştır, ters duranları değiştir" diye özetlenen ve neredeyse her programlama kitabında ilk öğretilen **kabarcık sıralaması** (bubble sort)?

Bu hafta üç temel sıralama algoritmasını, seçmeli, kabarcık ve araya ekleme sıralamasını, kâğıt üzerinde izleyip C, C++ ve Python'da yazacak, her karşılaştırmayı ve her eleman hareketini sayacağız. Haftanın deneyinde bu makinede ölçtüğümüz sürelerden yapılan kestirime göre bir milyon rastgele tamsayıyı C'de (-O2) kabarcık sıralaması ≈ 26,7 dk, araya ekleme sıralaması ≈ 3,8 dk sürede sıralar; Python'da kabarcık sıralaması ≈ 15,5 sa sürer. C kütüphanesinin `qsort` fonksiyonu aynı işi 153,9 ms sürede bitirir (ölçüm; Şekil 5.1). Aradaki ≈ 10 409 katlık fark daha hızlı bir bilgisayardan değil, büyüme hızından gelir: üç algoritmamız da $\Theta(n^2)$ işlem yapar, kütüphane ise $O(n \log n)$. Bu hafta "neden yavaş?" sorusunu ayrıntılandıracağız: hangi işlem kaç kez yapılıyor, hangi girdide hızlanıyor, eşit anahtarlı kayıtlara ne oluyor?

Şekil 5.1
$n = 10^6$ rastgele tamsayı, bu makinede: üç algoritmanın süre kestirimi (taralı, $n \le 20\ 000$ ölçümlerinden $T \propto n^2$) ve kütüphane sıralamasının ölçülen süresi; x ekseni logaritmik



GERÇEK DÜNYADAN · Google, Kasım 2007: "bir milyon 32 bitlik tamsayı"

Google'ın kamu politikası blogunda yayımlanan yazıya göre ABD başkan adayı Barack Obama, 14 Kasım 2007'de şirketin Mountain View'daki ana kampüsünü ziyaret eden yedinci aday oldu. Konuşmasının ardından sahnede Google'ın o zamanki CEO'su Eric Schmidt'le söyleşti. Yazının "standart bir Google mühendislik mülakatı sorusu" diye andığı soruyu Schmidt sordu: "Bir milyon 32 bitlik tamsayıyı sıralamanın en verimli yolu nedir?" Obama'nın yanıtı: "kabarcık sıralaması yanlış yol olur" ("the bubble sort would be the wrong way to go"; McLaughlin, 2007).

Kabarcık sıralamasının bu ünü yeni değildir. Astrachan (2003) algoritmanın kökenini "arkeolojik" bir çözümlemeyle inceler: makaleye göre "bubble sort" adı basılı olarak görünüşe göre ilk kez Iverson'ın 1962 tarihli kitabında geçer (yazar bunun kesin olmadığını belirtir); daha önce yöntem "değiş tokuşla sıralama"

(sorting by exchange) diye anılıyordu. Astrachan, Knuth'un yargısını da aktarır: kabarcık sıralamasının "akılda kalan bir adı ve bazı ilginç kuramsal sorulara yol açması dışında önerilecek hiçbir yanı yok gibidir" (Knuth, 1998, aktaran Astrachan, 2003). Aynı makalede Java'da dizgilerle kabarcık sıralaması araya eklemeden neredeyse üç kat yavaş çıkar; bu oranı kendi makinemizde ölçeceğiz.

1.1 Öğrenme çıktıları

- **ÖÇ 5.1** Seçmeli, kabarcık ve araya ekleme sıralamasını gerçekler; her biri için döngü değişmezini ifade eder. (Bölüm 4.1–4.3, Örnek 1–3)
- **ÖÇ 5.2** Üç algoritmayı karşılaştırma ve takas sayıları, en iyi/en kötü durum, kararlılık, yerindelik ve uyarlanabilirlik açısından karşılaştırır. (Bölüm 4.5, Bölüm 6, Örnek 4)
- **ÖÇ 5.3** Ters çevrim (inversion) kavramıyla araya ekleme sıralamasının çalışma süresini ilişkilendirir. (Bölüm 3.3, Örnek 5 ve 11)
- **ÖÇ 5.4** Karşılaştırıcı fonksiyonlarla yapı dizilerini çok anahtarlı ölçütlere göre sıralar (qsort, std::sort/stable_sort, sorted/key). (Bölüm 3.5–3.6, Örnek 7, 8 ve 10)
- **ÖÇ 5.5** Kararlılığın çok anahtarlı sıralamadaki rolünü deneyle gösterir. (Bölüm 3.4, Örnek 9 ve 10)

2. Geçen Haftadan ve AP-1'den Bağlantı

H4'te aramayı inceledik: sırasız dizide doğrusal arama Θ(n) yoklama ister; **sıralı** dizide ikili arama Θ(log n) ile yetinir. Bu ayrım bu haftanın varlık nedenidir: sıralama, aramayı ve daha birçok işlemi ucuzlatmak için bir kez ödenen bedeldir. H4'ün Örnek 8'inde sıralı bir diziye yeni gelen elemanı ikili aramayla bulduğumuz konuma, büyükleri birer sağa kaydırarak eklemiştik. Bu haftanın diliyle o program, **ikili aramalı araya ekleme sıralamasıydı**: yoklama sayısı küçüktü, ama kaydırma sayısı büyüktü.

AP-1'den iki yapı taşını yeniden kullanacağız. AP-1 H10'da bir dizinin **en küçüğünün indisini** bulmayı ve iki elemanı geçici değişkenle **takas** etmeyi (swap) yazdık; bu iki parça bu haftanın ilk algoritmasının gövdesidir (AP-1'de "seçme sıralaması" dediğimiz algoritmaya burada **seçmeli sıralama** diyoruz; ikisi aynı algoritmadır). AP-1 H14'te `struct` ile kayıt tanımladık; bu hafta öğrenci kayıtlarını ortalamaya, soyada ve ada göre sıralayacağız.

TAHMİN ET · **Geri çağırma: üç soru (yanıtlar aşağıda)**

- **S1 (H4).** 16 elemanlı sıralı `{3, 8, 14, …, 95}` dizisinde ikili arama x = 61'i en çok kaç yoklamada bulur? Dizi sıralı olmasaydı ne yapmamız gerekirdi?
- **S2 (H4, Örnek 8).** Rastgele 1000 sayıyı tek tek sıralı diziye eklediğimizde toplam kaydırma 253 576 çıktı; $n^2/4$ = 250 000. Bu oran (≈ 1,014) bir rastlantı mı?
- **S3 (AP-1 H10).** n elemanlı bir dizide en küçüğün indisini bulmak kaç karşılaştırma ister? Takası geçici değişken olmadan `a[i] = a[j]; a[j] = a[i];` diye yazarsak ne olur?

Yanıtlar. S1: 4 yoklama (H4 Örnek 3'ün izleme tablosu); sırasız dizide ya doğrusal arama yapar ya da önce sıralardık. S2: rastlantı değil. Bir sayının eklenirken yaptığı kaydırma, dizide ondan önce gelip ondan büyük olanların sayısıdır; rastgele sırada her çiftin "ters" olma olasılığı 1/2 olduğundan toplam $n(n-1)/4 \approx n^2/4$ civarındadır. Bu sayıya bu hafta **ters çevrim sayısı** diyeceğiz (Bölüm 3.3). S3: n − 1 karşılaştırma; geçici değişkensiz yazımda iki eleman da a[j]'nin değerini alır, a[i]'nin değeri kaybolur.

3. Kavramsal Anlatım: Sıralama Problemi ve Algoritma Özellikleri

3.1 Sıralama problemi ve neden sıraladığımız

Sıralama problemi (sorting problem): girdi n öğelik bir dizi a[0..n − 1] ve öğeler arasında bir "önce gelir" bağıntısıdır; çıktı, aynı öğelerin a[0] ≤ a[1] ≤ … ≤ a[n − 1] olacak biçimde yeniden dizilmiş hâlidir. Tanımın iki yarısı vardır ve ikisi de sınanmalıdır: çıktı **sıralı** olmalı ve çıktı girdinin bir **permütasyonu** olmalıdır (hiçbir öğe kaybolmamalı, çoğalmamalı). Bölüm 5.1'de yalnızca birinci koşulu sağlayan "sahte" bir sıralamanın testleri nasıl kandırdığını göreceğiz.

Neden sıralarız? Çünkü sıralı veride pek çok işlem tek geçişe iner: (1) **arama** ikili aramayla Θ(log n) olur (H4); (2) **yinelenen tespiti** komşuları karşılaştırmaya iner, çünkü eşit öğeler yan yana gelir; (3) **gruplama** ve sayma, eşitlerin ardışık bloklarını saymaktır; (4) iki sıralı dizi tek geçişte **birleştirilir** (H6'nın birleştirmeli sıralamasının çekirdeği). Kavram 4 (`kodlar/kavram/H05_K04_neden_siralariz.c`) bu dördünü aynı 12 sayılık dizide gösterir: sırasız dizide her çifti denetleyen yinelenen araması ilk yineleneni 20 karşılaştırmada bulurken, sıralı dizide komşu taraması (`a[k - 1] == a[k]`) 2 karşılaştırma yapar.

3.2 Neyi sayıyoruz? Karşılaştırma, takas ve kaydırma

Bu haftanın bütün örnekleri aynı **sayaç sözleşmesini** kullanır; sayılar ancak tanımları aynıysa karşılaştırılabilir:

- **Karşılaştırma:** iki dizi öğesinin (ya da bir öğe ile elde tutulan anahtarın) her karşılaştırması. Döngü sayaçlarının karşılaştırması (`i < n`) sayılmaz.
- **Takas** (swap): iki öğenin yer değiştirmesi. Seçmeli sıralamada takas yalnızca gerçekten yapıldığında (en küçük zaten yerindeyken yapılmaz) sayılır. "Takas" sözcüğünü yalnızca bu anlamda kullanıyoruz.
- **Kaydırma** (shift): araya eklemede bir öğenin bir konum sağa kopyalanması (`a[j + 1] = a[j]`); Shell sıralamasında h konum.
- **Yazma:** diziye yapılan öğe yazımı (deneyde): bir takas 2 yazma, araya eklemede her kaydırma 1 yazma ve her anahtarın yerine konması 1 yazmadır.

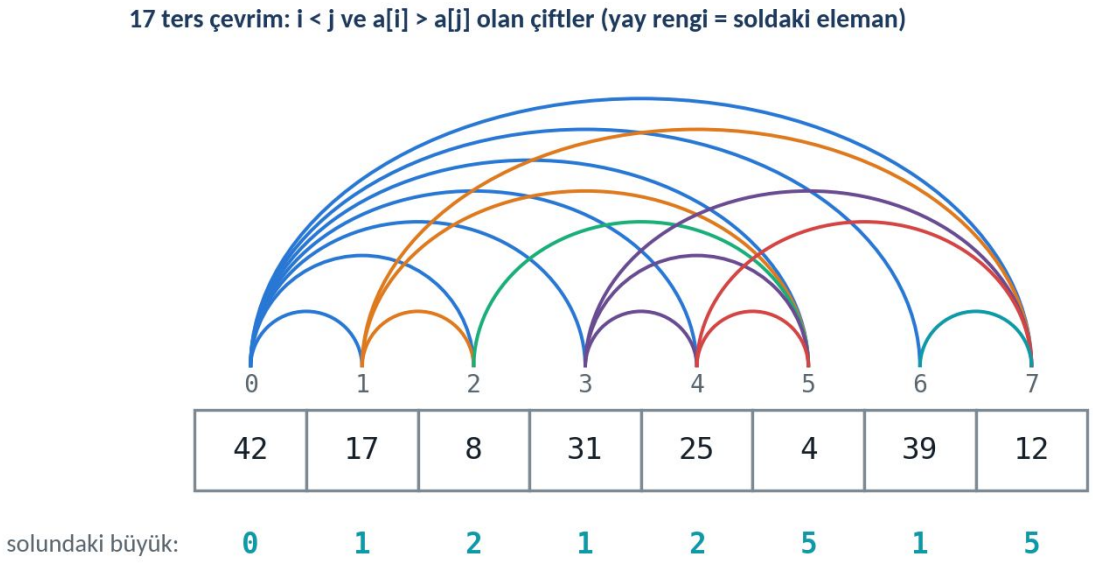
Karşılaştırma işlemcinin hesap ve dal (branch) maliyetini, takas ve kaydırma bellek trafiğini temsil eder. Öğeler küçük tamsayılarken ikisi de ucuzdur; öğeler 256 baytlık kayıtlarsa ya da karşılaştırma Türkçe bir dizgi karşılaştırmasıysa denge değişir (Bölüm 3.8).

3.3 Ters çevrim: bir dizinin "ne kadar sırasız" olduğu

Bir dizideki **ters çevrim** (inversion), i < j iken a[i] > a[j] olan bir (i, j) indis çiftidir: sırası yanlış olan bir çift. Haftanın ortak dizisi a = {42, 17, 8, 31, 25, 4, 39, 12} için 17 ters çevrim vardır (Şekil 5.2). Ters çevrim sayısı sıralı dizide 0, ters sıralı dizide en büyük değeri n(n − 1)/2'dir; rastgele bir permütasyonda her çift 1/2 olasılıkla ters olduğundan beklenen değer n(n − 1)/4'tür.

Ters çevrim sayısı bu haftanın gizli kahramanıdır, çünkü iki algoritmanın işini **tam olarak** ölçer. Komşu iki öğeyi takas etmek ters çevrim sayısını tam 1 azaltır: kabarcık sıralamasının takas sayısı ters çevrim sayısına eşittir. Araya eklemede bir öğe, solundaki kendisinden büyük her öğenin üstünden birer kez kaydırılarak geçer: kaydırma sayısı da ters çevrim sayısıdır. Örnek 5 tanımdan, her çifti denetleyerek (Θ(n^2)) sayar; ikinci bölümü her öğenin kaydırmasının solundaki büyüklerin sayısına eşit olduğunu tablo olarak yazar (Şekil 5.2 altındaki sayılar), üçüncü bölümü eşitliği rastgele dizilerde sınar:

Şekil 5.2
a = {42, 17, 8, 31, 25, 4, 39, 12} dizisinin 17 ters çevrimi: her yay bir (i, j) çifti; altta her elemanın solundaki kendisinden büyük öğe sayısı (Örnek 5)



Σ = 17 = araya ekleme kaydırması = kabarcık takası

c/H05_05_ters_cevrim.c · satır 20–31

```c
static long long ters_cevrim_say(const int *a, size_t n)
{
    long long sayi = 0;
    for (size_t i = 0; i < n; i++) {
        for (size_t j = i + 1; j < n; j++) {
            if (a[i] > a[j]) {
                sayi++;
            }
        }
    }
    return sayi;
}
```

ÇIKTI · ÖRNEK 5 (3. VE 4. BÖLÜM)

```
$ ./H05_05_ters_cevrim
3) rastgele diziler (değerler 0..999, ortak LCG)
   n  tohum  ters çevrim   kaydırma  eşit  oran (/ n(n-1)/4)
1000      1       254512     254512  evet  1.019
1000      2       239605     239605  evet  0.959
1000      3       243630     243630  evet  0.975
1000      4       254550     254550  evet  1.019
tüm denemelerde kaydırma = ters çevrim: evet
4) sınırlar (n = 1000): sıralı -> 0, ters sıralı -> 499500 (n(n-1)/2 = 499500)
```

Tablonun son sütunu ölçülen ters çevrim sayısının n(n − 1)/4 beklenen değerine oranıdır ve 1'e yakındır. Bu nedenle araya ekleme sıralamasının süresi n + (ters çevrim sayısı) ile orantılıdır: sıralı veride

doğrusal, rastgele veride karesel (Bölüm 4.3). Ters çevrimleri burada $\Theta(n^2)$'de saydık; H6'da birleştirmeli sıralamayı biraz değiştirerek O(n log n)'de sayacağız.

3.4 Algoritma özellikleri: kararlı, yerinde, uyarlanabilir, çevrimiçi

Aynı karmaşıklık sınıfındaki iki sıralama algoritması dört özellikte ayrışabilir:

- **Kararlı** (stable): anahtarları eşit olan öğelerin girdideki göreli sırası çıktıda korunur. Tamsayı dizisinde fark görünmez; ama ortalamaya göre sıraladığımız öğrenci kayıtlarında, ortalaması eşit olanların kayıt sırasında kalıp kalmadığı önemlidir.
- **Yerinde** (in-place): girdinin dışında yalnızca O(1) ek bellek kullanılır (birkaç değişken). Bu haftanın dört algoritması da yerindedir.
- **Uyarlanabilir** (adaptive): girdi sıralıya ne kadar yakınsa algoritma o kadar az iş yapar; ölçütü genellikle ters çevrim sayısıdır.
- **Çevrimiçi** (online): veri parça parça gelirken, gelen her öğe o ana kadarkilerin sıralı tutulduğu yapıya eklenebilir; algoritmanın bütün girdiyi baştan görmesi gerekmez.

Tablo 5.1 dört algoritmayı bu özelliklere göre karşılaştırır; gerekçeleri Bölüm 4'te her algoritmanın analiz adımındadır.

Tablo 5.1
Dört algoritmanın özellikleri

Özellik	Seçmeli	Kabarcık (erken çıkışlı)	Araya ekleme	Shell (merak)
Kararlı	hayır (uzak takas)	evet (eşitte takas yok)	evet (eşitte dur)	hayır (uzak kaydırma)
Yerinde (O(1) ek bellek)	evet	evet	evet	evet
Uyarlanabilir	hayır: her girdide n(n − 1)/2 karşılaştırma	zayıf: sıralıda n − 1 karşılaştırma, ama tek bir "kaplumbağa" n − 1 geçiş ister	evet: Θ(n + ters çevrim)	kısmen
Çevrimiçi	hayır	hayır	evet	hayır
Takas / kaydırma	en çok n − 1 takas	takas = ters çevrim	kaydırma = ters çevrim	kaydırma ≤ ters çevrim (uzak kaydırma birden çok ters çevrim giderir)

Şekil 5.3 altı kaydı gösterir: sayı sıralama anahtarı, harf girdideki sıradır. Seçmeli sıralama 2a ile 2b'yi ve 3a ile 3b'yi ters çevirir: bu bir hata değildir, çıktı anahtara göre sıralıdır; yalnızca kararlılık güvencesi yoktur. **Kararlılık deneyi (Örnek 9).** Örnek 8'in on öğrenci kaydı (Bölüm 5.2) üç algoritmayla ortalamaya göre azalan sıralanır; program, ortalaması eşit olup kayıt sırası bozulan çiftleri listeler. Programın seçmeli sıralaması Bölüm 4.1'dekinin kayıt sürümüdür (`ogrenci_takas(&a[i], &a[en])`); **uzak takas** eşitleri birbirinin üstünden atlatır:

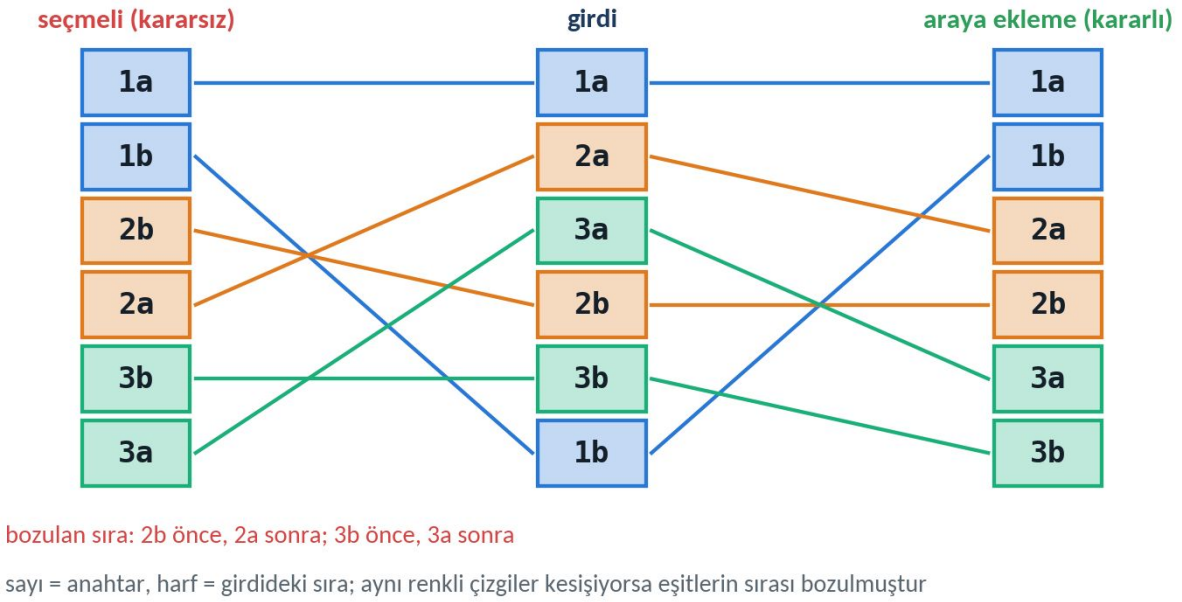
ÇIKTI · ÖRNEK 9 (KESİT)

```
$ ./H05_09_kararlilik
seçmeli:
    90.00(104) 90.00(108) 78.50(103) 78.50(101) 78.50(105)
    78.50(107) 78.50(110) 65.25(102) 65.25(109) 65.25(106)
   bozulan eşit çiftler:
    (103 önce, 101 sonra) (109 önce, 106 sonra)
```

```
kararlı davrandı mı? HAYIR (2 çift)
(kabarcık ve araya ekleme:)

kararlı davrandı mı? evet (0 çift)
```

Şekil 5.3
Kararlılık: anahtar (sayı) ve girdi sırası (harf) taşıyan altı kayıt. Seçmeli sıralamanın uzak takasları eşit anahtarlıların sırasını bozar (kesişen aynı renkli çizgiler); araya ekleme korur



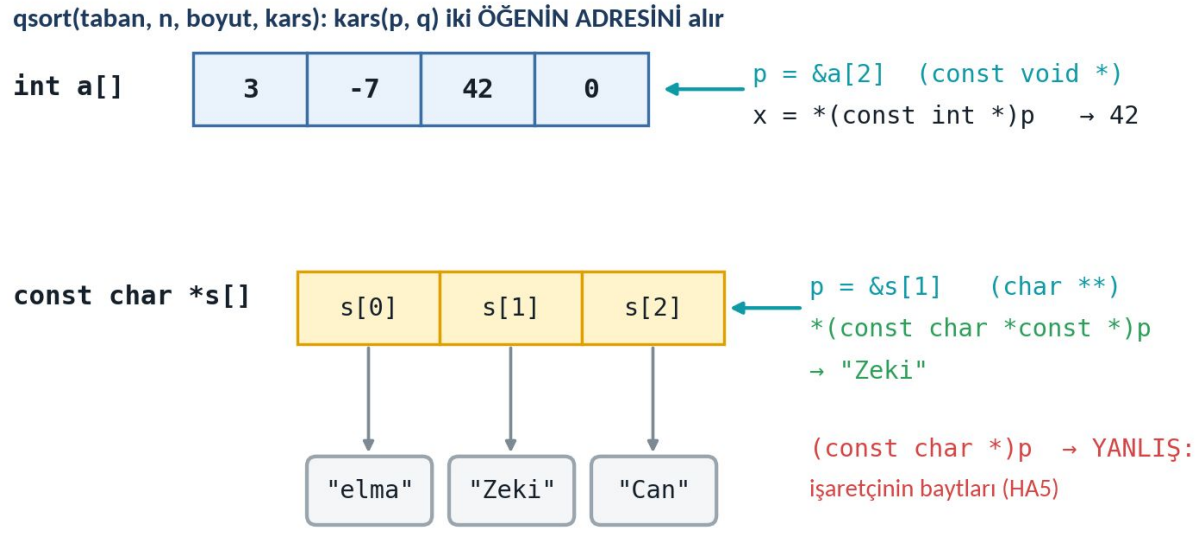
Seçmeli sıralamanın ilk geçişinde en büyük ortalama (90,00, no 104) a[0]'a takas edilir; a[0]'daki 101 numaralı kayıt (78,50) a[3]'e, yani öteki 78,50'lilerin arasına fırlatılır ve 103'ün arkasına düşer. Kabarcıkta eşitte takas, araya eklemede eşitte kaydırma yoktur; ikisi de bozulan çift bulmaz. Kararlılık bir algoritmanın **güvencesidir**, belirli bir girdide gözlenen bir davranış değil: kararsız bir algoritma bazı girdilerde eşitleri korumuş görünebilir.

3.5 Karşılaştırıcı sözleşmesi ve genel sıralama arayüzü

Bir sıralama algoritması öğelerin **ne** olduğunu bilmek zorunda değildir; yalnızca iki öğeden hangisinin önce geleceğini sorabilmelidir. Kütüphaneler bu soruyu bir **karşılaştırıcıya** (comparator) devreder. C'nin standart kütüphanesindeki sıralama fonksiyonu `void qsort(void *taban, size_t n, size_t boyut, int (*kars)(const void *, const void *))` biçimindedir: dizinin başlangıç adresini, öğe sayısını, bir öğenin bayt cinsinden boyutunu ve karşılaştırıcıyı alır. Karşılaştırıcı iki öğenin **adresini** `const void *` olarak alır ve birinci öğe önce gelecekse negatif, sonra gelecekse pozitif, eşitse 0 döndürür (Şekil 5.4).

Örnek 7'nin karşılaştırıcıları iki tuzaktan kaçınır. Birincisi **taşma**dır: `return x - y;` kısa ve doğru görünür, ama x = INT_MAX, y = −1 için fark `int`'e sığmaz; işaretli tamsayı taşması C'de **tanımsız davranıştır** (undefined behavior; Hata Avcısı 3). Güvenli deyiş `(x > y) - (x < y)`'dir: iki karşılaştırmanın sonucu 0 ya da 1'dir, farkları −1, 0 ya da 1 olur. İkincisi **kesme**dir: `double` için `(int)(x - y)` 0,1 ile 0,3'ü "eşit" sayar. Azalan sıra için bağımsız değişkenlerin yeri değiştirilir; dizgi dizisinde öğe bir `char *` olduğundan karşılaştırıcı önce adresi `const char *const *`'a çevirip işaretçiyi okur (`dizgi_artan`, Örnek 7'nin 3. bölümü):

Şekil 5.4
*qsort karşılaştırıcı sözleşmesi: p ve q öğelerin adresidir; dizgi dizisinde öğe bir char * olduğundan p aslında char ** gösterir*



c/H05_07_qsort_arayuzu.c · satır 20–31

```c
static int int_artan(const void *p, const void *q)
{
    int x = *(const int *)p;
    int y = *(const int *)q;
    return (x > y) - (x < y);   /* -1, 0, 1; taşma yok */
    /* YANLIŞ: return x - y;  x = INT_MAX, y = -1 için taşar (UB) */
}

static int int_azalan(const void *p, const void *q)
{
    return int_artan(q, p);     /* bağımsız değişkenlerin yeri değişir */
}
```

ÇIKTI · ÖRNEK 7 (1. BÖLÜM)

```
$ ./H05_07_qsort_arayuzu
1) int dizisi: qsort + taşmaya güvenli karşılaştırıcı
   girdi : 3 -7 2147483647 0 -2147483648 42 -1 7
   artan : -2147483648 -7 -1 0 3 7 42 2147483647
   azalan: 2147483647 42 7 3 0 -1 -7 -2147483648
   neden x - y değil? INT_MAX - (-1) = 2147483648 > INT_MAX = 2147483647
```

C++'ta `std::sort` üç değerli bir tamsayı değil, "x, y'den **önce** mi?" sorusunu yanıtlayan bir `bool` **yüklem** (predicate) alır; varsayılanı `x < y`'dir ve azalan sıra için `std::greater<int>()` verilir. Yüklem bir **katı zayıf sıralama** (strict weak ordering) tanımlamalıdır: `<` yerine `<=` yazmak bu koşulu bozar ve tanımsız davranıştır. Python'da çoğunlukla karşılaştırıcı değil bir **anahtar fonksiyonu** (`key=`) verilir; her öğe için bir kez hesaplanır ve sonuçlar karşılaştırılır. C tarzı bir karşılaştırıcı gerekirse `functools.cmp_to_key` ile anahtara çevrilir (`sorted(a, key=functools.cmp_to_key(int_artan))`; Örnek 7'nin C++ ve Python sürümleri Ek A.7'de).

Bu örnek kütüphane sıralamasını **kullanmanın** örneğidir; kendi algoritmalarımızı yazarken (Örnek 1–6, 8–13) kütüphaneyi yalnızca kâhin olarak kullanıyoruz. Kütüphanelerin hangi güvenceyi verip hangisini vermediği Bölüm 9'un konusudur.

3.6 Çok anahtarlı sıralama ve Türkçe alfabe

Gerçek listelerde tek anahtar nadiren yeter: başarı listesinde önce ortalama (büyükten küçüğe), ortalaması eşitlerde soyad, o da eşitse ad (A'dan Z'ye) sıralanır. Buna **sözlük sırası** (lexicographic order) denir ve iki yolla kurulur. **Tek karşılaştırıcı:** anahtarlar önem sırasıyla denenir; ilk farklı anahtar kararı verir. **Kararlı ardışık geçişler:** önce en önemsiz anahtara (ad), sonra soyada, en son ortalamaya göre **kararlı** bir algoritmayla sıralanır. Son geçiş ortalamaya göre dizer; ortalaması eşitler kararlılık sayesinde bir önceki geçişin (soyad, ad) sırasını korur (Şekil 5.5).

Şekil 5.5
Örnek 10'un on kaydında kararlı ardışık geçişler: ad → soyad → ortalama. Son sütun tek karşılaştırıcılı sıralamanın sonucuyla aynıdır (program denetler)

girdi (kayıt sırası)		1) ad ↑		2) soyad ↑ (kararlı)		3) ortalama ↓ (kararlı)	
101 Ayşe Yılmaz	78,50	107 Ali Yılmaz	78,50	108 Deniz Arslan	90,00	108 Deniz Arslan	**90,00**
102 Mehmet Kaya	65,25	101 Ayşe Yılmaz	78,50	106 Burak Demir	65,25	104 Can Yılmaz	**90,00**
103 Zeynep Demir	78,50	106 Burak Demir	65,25	110 Selin Demir	78,50	110 Selin Demir	**78,50**
104 Can Yılmaz	90,00	104 Can Yılmaz	90,00	103 Zeynep Demir	78,50	103 Zeynep Demir	**78,50**
105 Elif Kaya	78,50	108 Deniz Arslan	90,00	105 Elif Kaya	78,50	105 Elif Kaya	**78,50**
106 Burak Demir	65,25	105 Elif Kaya	78,50	109 Emre Kaya	65,25	107 Ali Yılmaz	**78,50**
107 Ali Yılmaz	78,50	109 Emre Kaya	65,25	102 Mehmet Kaya	65,25	101 Ayşe Yılmaz	**78,50**
108 Deniz Arslan	90,00	102 Mehmet Kaya	65,25	107 Ali Yılmaz	78,50	106 Burak Demir	**65,25**
109 Emre Kaya	65,25	110 Selin Demir	78,50	101 Ayşe Yılmaz	78,50	109 Emre Kaya	**65,25**
110 Selin Demir	78,50	103 Zeynep Demir	78,50	104 Can Yılmaz	90,00	102 Mehmet Kaya	**65,25**

son sıra = tek karşılaştırıcı (ortalama ↓, soyad ↑, ad ↑): eşit ortalamalılar önceki geçişin (soyad, ad) sırasını korur

Örnek 10 dört yolu karşılaştırır. **A:** tek karşılaştırıcı (araya ekleme, bir geçiş). **B:** kararlı araya eklemeyle üç ardışık geçiş. **C:** aynı geçişler kararsız seçmeli sıralamayla. **D:** tek (tam) karşılaştırıcıyla kararsız seçmeli sıralama. B, aynı araya ekleme fonksiyonunu üç kez çağırır: `ad_artan`, `soyad_artan`, `ortalama_azalan`. Tek karşılaştırıcı ise anahtarları önem sırasıyla dener; ilk sıfırdan farklı sonuç kararı verir:

c/H05_10_cok_anahtarli.c · satır 40–52

```c
/* Tek karşılaştırıcı: sözlük sırası (önce en önemli anahtar) */
static int cok_anahtarli(const Ogrenci *x, const Ogrenci *y)
{
    int k = ortalama_azalan(x, y);
    if (k != 0) {
        return k;
    }
    k = soyad_artan(x, y);
    if (k != 0) {
        return k;
```

```
50      }
51      return ad_artan(x, y);
52  }
```

ÇIKTI · ÖRNEK 10 (B VE C)

```
$ ./H05_10_cok_anahtarli
B) kararlı ardışık geçişler: ad -> soyad -> ortalama
   B sırası: 108 104 110 103 105 107 101 106 109 102
   A ile aynı mı? evet
   karşılaştırma: A = 32 (1 geçiş), B = 88 (3 geçiş)

C) aynı ardışık geçişler, kararsız seçmeli sıralamayla:
   C sırası: 108 104 110 103 105 107 101 109 102 106
```

B'nin sırası A ile aynıdır, ama üç geçiş daha çok karşılaştırma yapar (88'e karşı 32): ardışık geçişlerin değeri, karşılaştırıcıyı değiştirmeden yeni bir ölçüt eklemeye izin vermeleridir. C'nin sırası farklıdır: kararsız geçiş bir önceki geçişin sırasını bozar. D (tek, tam karşılaştırıcıyla kararsız seçmeli) yine A ile aynıdır: tam karşılaştırıcıyla "eşit" kalan öğe yoktur, kararlılık gerekmez. Ardışık geçiş fikri H7'de karşımıza çıkacak: taban sıralaması (radix sort) sayıları basamak basamak, en önemsiz basamaktan başlayarak kararlı geçişlerle sıralar.

Şekil 5.6
Bayt sırası ve Türkçe alfabe sırası: UTF-8'de Ç (C3 87) ve Ş (C5 9E) bütün ASCII harflerinden büyük baytlarla başlar; strcmp onları listenin sonuna atar (Hata Avcısı 8)

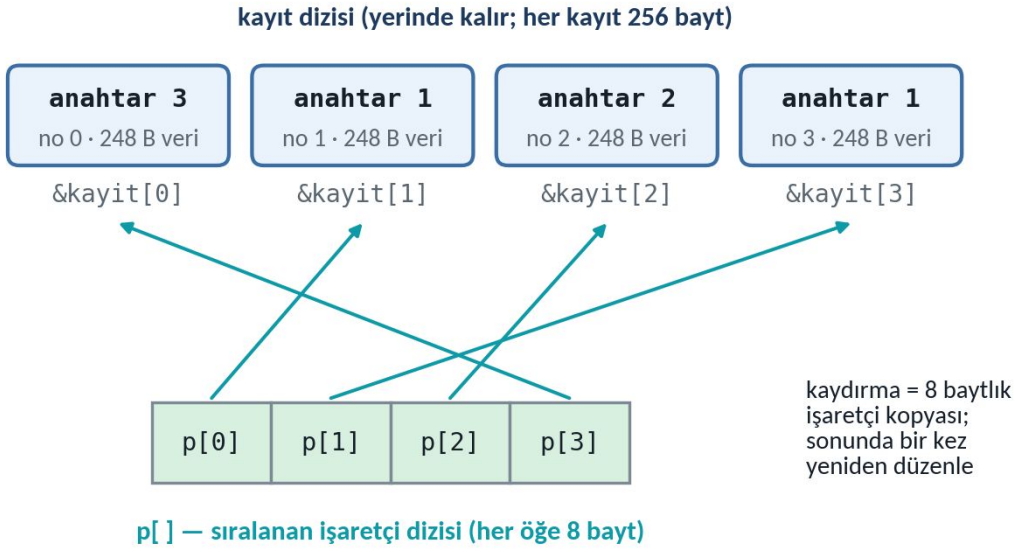


Dizgi anahtarlarında bir tuzak daha var. C'de `strcmp`, C++'ta `std::string`'in `<` işleci ve Python'da `str` karşılaştırması baytları (Python'da kod noktalarını) karşılaştırır. UTF-8'de Türkçe Ç, Ö, Ü, Ş, Ğ, İ harfleri iki baytla yazılır ve ilk baytları bütün ASCII harflerinden büyüktür; bu yüzden bayt sırasında "Çelik", "Zeki"den sonra gelir: Ç UTF-8'de C3 87, Ş C5 9E, Z ise tek bayt 5A'dır (Şekil 5.6; Hata Avcısı 8). Örnek 10'un adlarında ilk farklı harfler ASCII'dir, bu yüzden orada bayt sırası alfabe sırasıyla aynıdır. Türkçe alfabe sırası için harfler kod noktasına çözülüp bir sıra tablosuyla karşılaştırılmalıdır; Mini-OBS 2 v2.4'ün `tr_karsilastir` fonksiyonu bunu yapar (Bölüm 10).

3.7 Dolaylı sıralama: kayıtları değil işaretçileri taşımak

Kayıtlar büyükse (ad, adres, notlar… her biri yüzlerce bayt) her takas ve kaydırma büyük bir bellek kopyasıdır. **Dolaylı sıralama** (indirect sort) kayıtları yerinde bırakır ve onları gösteren bir **işaretçi dizisini** (Python'da indis listesini) sıralar; sıralı sıra gerekiyorsa kayıtlar sonunda bir kez yeniden düzenlenir (Şekil 5.7). Öğe kopyası sayısı aynıdır, ama kopyalanan bayt sayısı kayıt boyutu ile işaretçi boyutunun oranı kadar azalır. Ayrıca aynı kayıtlar üzerinde aynı anda birçok sıralı "görünüm" tutulabilir: biri ortalamaya, biri soyada göre. Mini-OBS 2'nin listeleri böyle kurulur.

Şekil 5.7
Dolaylı sıralama: 256 baytlık kayıtlar yerinde kalır; anahtara göre (kararlı) sıralanan, 8 baytlık işaretçilerden oluşan p[] dizisidir



Örnek 12 (n = 1000): kopyalanan bayt 61 829 120 (kayıtlar) ↔ 2 188 160 (işaretçiler + düzenleme), 28,3 kat

Örnek 12 1000 kaydı (her biri 256 bayt; `_Static_assert` boyutu derleme zamanında denetler; anahtarlar 0..99, yani bol yinelenen) iki yolla araya eklemeyle sıralar: doğrudan sürüm kayıtları, dolaylı sürüm işaretçileri kaydırır ve sonunda kayıtları bir kez yeniden düzenler:

c/H05_12_dolayli_siralama.c · satır 49–63

```c
static void isaretci_sirala(const Kayit **p, size_t n)
{
    for (size_t i = 1; i < n; i++) {
        const Kayit *anahtar = p[i];
        kopya++;
        ptrdiff_t j = (ptrdiff_t)i - 1;
        while (j >= 0 && p[j]->anahtar > anahtar->anahtar) {
            p[j + 1] = p[j];
            kopya++;
            j--;
        }
        p[j + 1] = anahtar;
        kopya++;
    }
}
```

ÇIKTI · ÖRNEK 12 (KESİT)

```
$ ./H05_12_dolayli_siralama
yöntem                          öğe kopyası  kopyalanan bayt
doğrudan: kayıtları kaydır           241520       61829120
dolaylı: işaretçileri kaydır         241520        1932160
  + bir kez yeniden düzenleme          1000         256000
dolaylı toplam                       242520        2188160
kopyalanan bayt oranı (doğrudan / dolaylı toplam) = 28.3
iki yöntem aynı kayıt sırasını verdi mi? evet
```

3.8 Kaputun Altında: bellek trafiği, memmove ve melez kütüphaneler

KAPUTUN ALTINDA · **Takas maliyeti ve büyük yapılar**

Örnek 12'nin iki yöntemi aynı sayıda öğe kopyası yapar; kopyalanan bayt oranı (kayıtlar / işaretçiler + bir kez düzenleme) n = 1000'de 28,3, n = 5 000'de 31,2'dir. Bu makinede n = 5 000 için tek bir çalıştırmada süre C'de 55,9 ms'den 8,8 ms'ye, C++'ta 51,6 ms'den 19,7 ms'ye indi (oranlar 6,4 ve 2,6). Süre bayt oranı kadar düşmez: büyük ardışık kopyalar hızlıdır, işaretçi sürümünde ise her karşılaştırma bir işaretçi izleyip belleğin başka bir yerine gider. Python'da sonuç tersine döndü (1 288 ms'ye karşı 2 302 ms): Python listesi zaten nesnelere **başvuru** tutar, yani "doğrudan" sıralama da yalnızca başvuruları taşır. Aynı fikrin değeri dile göre değişir: ölçmeden karar vermeyin.

KAPUTUN ALTINDA · **Gerçek kütüphaneler küçük parçalarda araya ekleme kullanır**

$\Theta(n^2)$ bir algoritma büyük n için yanlış yoldur, küçük n için ise en hızlı yol olabilir: döngüsü kısadır, ek belleği yoktur, sıralıya yakın veride doğrusaldır. Bu yüzden gerçek kütüphaneler **melez** (hybrid) çalışır: büyük diziyi $\Theta(n \log n)$ bir yöntemle parçalara ayırır, küçük parçaları araya eklemeyle sıralar. C++ kütüphanelerinin `std::sort`'u genellikle introsort'tur (Musser, 1997); Python'un `list.sort`'u Timsort tabanlıdır. Hata Avcısı 7'deki gözlem (bu kütüphanenin `std::sort`'u 16 öğede eşitleri korudu, 24 öğede bozdu) küçük girdide araya ekleme çalıştığını düşündürür ama kanıtlamaz. Kendi melez sıralamamızı H6'da kuracağız.

KAPUTUN ALTINDA · **Kaydırmayı blok olarak yapmak: memmove**

Araya eklemenin iç döngüsü ardışık bellekte sağdan sola ilerler: her kaydırma bir öncekinin hemen yanındaki hücreye yazar. Bu erişim deseni önbellek dostudur (her önbellek satırı birkaç öğe getirir). Bir adım ileri gidilebilir: anahtarın yeri önce yalnızca karşılaştırarak bulunur, sonra aradaki blok tek bir `memmove` çağrısıyla bir sağa taşınır. Bölgeler örtüştüğü için `memcpy` değil `memmove` kullanılmalıdır; `memcpy`'nin örtüşen bölgelerde davranışı tanımsızdır.

kavram/H05_K01_memmove_kaydirma.c · satır 36–50

```
36  /* Blok: önce yeri bul, sonra a[yer..i-1]'i tek memmove ile kaydır */
37  static void araya_ekleme_memmove(int *a, size_t n)
38  {
39      for (size_t i = 1; i < n; i++) {
40          int anahtar = a[i];
41          size_t yer = i;
42          while (yer > 0 && a[yer - 1] > anahtar) {
43              yer--;                      /* yalnızca karşılaştır */
44          }
45          /* bölgeler örtüşür: memcpy değil memmove */
46          memmove(&a[yer + 1], &a[yer], (i - yer) * sizeof a[0]);
47          kaydirma += (long long)(i - yer);
```

```
48          a[yer] = anahtar;
49      }
50  }
```

Kavram 1 iki sürümün aynı sonucu ve aynı kaydırma sayısını verdiğini denetler. n = 20 000'de tek bir çalıştırmada süre rastgele girdide 90,3 ms'den 53,0 ms'ye, ters sıralı girdide 186,1 ms'den 76,8 ms'ye indi (stderr; bu makinede, -O2). Karmaşıklık değişmedi (hâlâ $\Theta(n^2)$ kaydırma), yalnızca sabit küçüldü: kütüphanenin `memmove`'u büyük blokları geniş yazmaçlarla kopyalar.

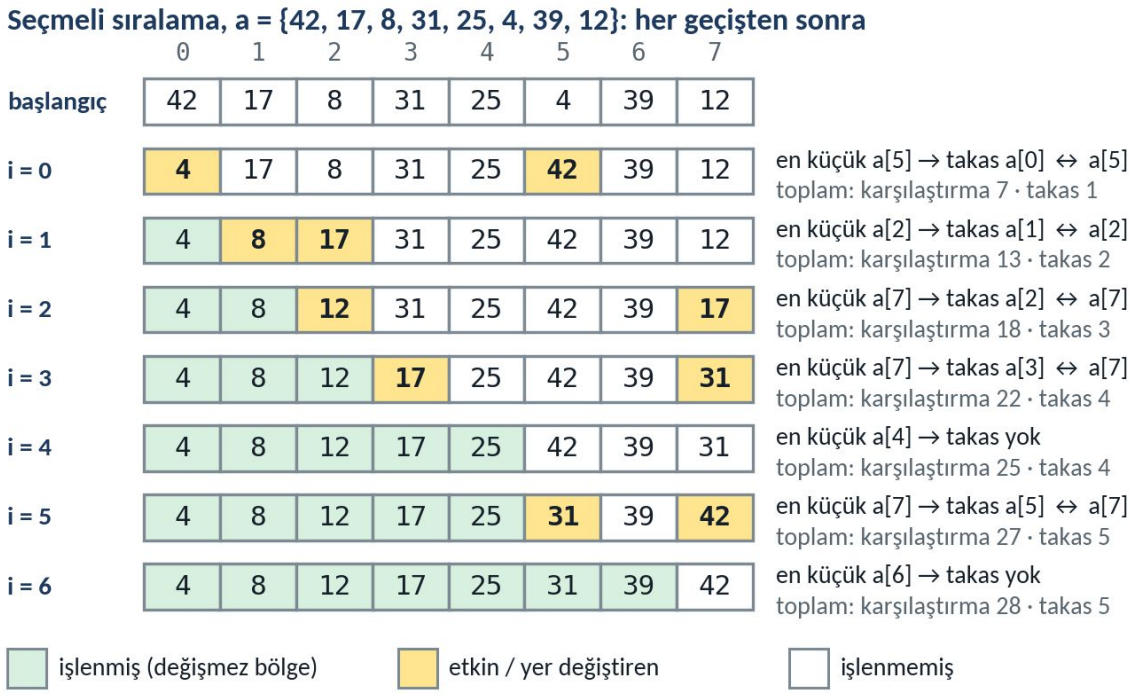
4. Algoritmalar

Üç amiral algoritmayı ve merak kutusundaki Shell sıralamasını aynı on bir adımla (problem, fikir, elle izleme, sözde kod, doğruluk, analiz ve kimlik kartı, C, üç dil, test, deney, varyantlar) ve aynı dizide işliyoruz: a = {42, 17, 8, 31, 25, 4, 39, 12} (n = 8, 17 ters çevrim). Bölüm 4.5'teki Şekil 5.13 her birinin değişmezinin söz verdiği bölgeyi ayırır, Şekil 5.14 üçünü aynı anlarda çubuklarla yan yana gösterir; web sayfasındaki sıralama görselleştiricisi aynı adımları canlandırır. Üç algoritmanın çözümlemesi standart kaynaklardakiyle aynıdır (Cormen vd., 2022; Knuth, 1998); biz her iddiayı ayrıca sayaçlarla sınayacağız.

4.1 Seçmeli sıralama (selection sort)

1 · Problem. a[0..n − 1] dizisini artan sıraya dizmek; örnek girdi a = {42, 17, 8, 31, 25, 4, 39, 12}. **2 · Fikir.** Sınıfı boy sırasına dizen bir öğretmen gibi: kalanların en kısasını bul, sıranın başına çağır; kalanlar arasında yine en kısayı bul, ikinci sıraya çağır… Her geçişte kalan kısmın **en küçüğünün indisi** bulunur (AP-1 H10) ve öneğin sonuna **takas** edilir. Sıralı önek her geçişte bir öğe büyür.

Şekil 5.8
Seçmeli sıralama, a = {42, 17, 8, 31, 25, 4, 39, 12}: her geçişten sonra dizi (sarı: takas edilen iki öğe; yeşil: değişmezin önek bölgesi) ve o ana kadarki sayaçlar



3 · Elle izleme. Şekil 5.8 her geçişi gösterir. i = 0'da kalan kısmın en küçüğü a[5] = 4'tür; a[0] ile takas edilir. i = 2 ile i = 3'te en küçük dizinin sonundadır (12 ve 17). Toplam 28 karşılaştırma ve 5 takas yapılır.

> **TAHMİN ET · Sizce i = 4 geçişinde ne olur?**
>
> i = 3'ten sonra dizi {4, 8, 12, 17, 25, 42, 39, 31}. i = 4 geçişinde en küçük hangi indiste bulunur, takas yapılır mı? (Yanıt: en küçük a[4] = 25'tir, yani zaten yerindedir; `en_kucuk != i` koşulu yanlış olduğundan takas yapılmaz. Bu yüzden takas sayısı n − 1 = 7 değil 5'tir.)

SÖZDE KOD · 4 · Sözde kod: seçmeli sıralama

```
algoritma SEÇMELİ-SIRALA(a[0..n−1])
    i ← 0'dan n−2'ye kadar her i için
        en ← i                                  ▷ kalanların en küçüğünün indisi
        j ← i+1'den n−1'e kadar her j için
            eğer a[j] < a[en] ise en ← j
        eğer en ≠ i ise a[i] ile a[en]'i takas et
```

5 · Doğruluk. Değişmez (invariant; Tablo 5.2): i. geçişin sonunda a[0..i] sıralıdır ve dizinin en küçük i + 1 elemanını içerir (a[0..i] ≤ a[i+1..n−1]). **Başlatma:** ilk geçişten önce önek boştur, söz kendiliğinden doğrudur. **Koruma:** i. geçiş kalan kısmın (a[i..n − 1]) en küçüğünü a[i]'ye koyar; bu öğe önekteki her öğeden büyük ya da eşittir (önek kalanlardan küçükleri içerir) ve kalanların her birinden küçük ya da eşittir; önek bir öğe uzar ve söz korunur. **Sonlanma:** döngü i = n − 2'den sonra biter; a[0..n − 2] en küçük n − 1 öğeyi sıralı içerir, geriye kalan tek öğe en büyüktür ve dizi sıralıdır. Örnek 1, değişmezi her geçişten sonra `degismez_tamam_mi` ile programda da denetler.

6 · Analiz. i. geçiş tam n − 1 − i karşılaştırma yapar; toplam (n − 1) + (n − 2) + … + 1 = n(n − 1)/2. Bu sayı **girdiden bağımsızdır**: sıralı, ters ya da rastgele dizide aynıdır, bu yüzden en iyi, ortalama ve en kötü durum $\Theta(n^2)$'dir; seçmeli sıralama uyarlanabilir değildir. Takas sayısı ise en çok n − 1'dir (her geçişte en çok bir); bu, seçmeli sıralamanın tek güçlü yanıdır: yazma pahalıysa en az eleman hareketi yapan temel algoritmadır. Ek bellek O(1). Uzak takas (a[i] ↔ a[en]) arada kalan eşit bir öğenin üstünden atlayabildiği için kararsızdır (Örnek 9).

Tablo 5.2
Seçmeli sıralama kimlik kartı

ALGORİTMA KİMLİK KARTI **Seçmeli sıralama** *(selection sort)*	
Strateji	Artımlı (sıralı önek her geçişte bir eleman büyür): kalan kısmın en küçüğünü seç, öneğin sonuna takasla
Girdi önkoşulu	Yok — karşılaştırılabilir anahtarlar; rastgele erişimli dizi
Zaman (en iyi · ortalama · en kötü)	$\Theta(n^2)$ · $\Theta(n^2)$ · $\Theta(n^2)$
Ek bellek	O(1)
Özellikler	Kararsız · yerinde · uyarlanabilir değil · en fazla n − 1 takas. Karşılaştırma sayısı her girdide tam n(n−1)/2 (n = 1000: 499 500); takas ters sıralıda ⌊n/2⌋ (n = 1000: 500), sıralıda 0
Değişmez / doğruluk	i. geçişin sonunda a[0..i] sıralıdır ve dizinin en küçük i + 1 elemanını içerir (a[0..i] ≤ a[i+1..n−1])
Kütüphane karşılığı	C: yok (elle); genel sıralama `qsort` · C++: yok; yapı taşları `std::min_element` + `std::iter_swap`; genel `std::sort` · Python: yok; genel `sorted(a)`, `a.sort()` (yalnızca kâhin)
Ne zaman kullanılır	Yazma/takas çok pahalıysa (en fazla n − 1 takas); çok küçük n; öğretimde ve kâhin olarak

Ne zaman kullanılmaz	Büyük n (her girdide $\Theta(n^2)$ karşılaştırma); kararlılık gerektiğinde; neredeyse sıralı veride (uyarlanabilir değildir)

7 · C gerçekleştirimi. (Örnek 1; sayaçlar dosya düzeyi `static` değişkenlerdir, `takas` yardımcısı geçici değişkenle yazılır):

c/H05_01_secmeli_siralama.c · satır 66–84

```c
static void secmeli_sirala(int *a, size_t n, int yazdir)
{
    for (size_t i = 0; i + 1 < n; i++) {
        size_t en_kucuk = i;
        for (size_t j = i + 1; j < n; j++) {
            karsilastirma++;
            if (a[j] < a[en_kucuk]) {
                en_kucuk = j;
            }
        }
        if (en_kucuk != i) {
            takas(&a[i], &a[en_kucuk]);
            takas_sayisi++;
        }
        if (yazdir) {
            gecis_yaz(a, n, i, en_kucuk);
        }
    }
}
```

Program sonunda karşılaştırma sayısını (28; n(n − 1)/2 = 28) ve takas sayısını (5; en fazla n − 1 = 7) yazar. Dış döngü koşulu `i < n - 1` değil `i + 1 < n` yazılmıştır: `size_t` işaretsizdir ve n = 0 iken `n - 1` çok büyük bir sayıya sarar; programın son iki satırı n = 0 ve n = 1 için sıfır karşılaştırma yapıldığını gösterir.

8 · Üç dil karşılaştırması. Algoritma üç dilde satır satır aynıdır; anlamca farklılaşan satırlar azdır. C++'ta dizi bir `std::vector<int>&` referansıdır (`n = a.size()`) ve takas `std::swap(a[i], a[en_kucuk])`'tır (`H05_01_secmeli_siralama.cpp`, satır 76). Python'da takas çoklu atamadır, `a[i], a[en_kucuk] = a[en_kucuk], a[i]` (satır 56); dosya düzeyindeki sayaçlar fonksiyonda `global` bildirimi ister ve iç döngü `range(i + 1, n)` ile yazılır. İki sürümün tam fonksiyonları ve fark notu Ek A.1'dedir.

9 · Test. Sınır durumları n = 0, n = 1, hepsi eşit, sıralı ve ters sıralı dizilerdir. Örnek 6 bunları ve n = 0 … 30 için rastgele dizileri kendi sıralamamıza ve **kâhine** (test oracle; burada kütüphane sıralaması `qsort`) verip sonuçları karşılaştırır: seçmeli sıralamanın çıktısı 628/628 dizide sıralı, 628/628 dizide kâhinle aynıdır. **10 · Deney.** n = 10 000'de karşılaştırma sayısı dört girdi türünde de 49 995 000'dir (Bölüm 6); C'de süreler 108,0 ms, 109,1 ms, 110,3 ms, 107,4 ms ile birbirine çok yakındır: seçmeli sıralama girdiye körtür. **11 · Varyantlar.** Her geçişte en büyüğü bulup sona koymak (Alıştırma 5.2); azalan sıra (Alıştırma 5.11); her geçişte hem en küçüğü hem en büyüğü yerleştirmek (geçiş sayısı yarıya iner, karşılaştırma sayısı ise yaklaşık aynı kalır); kayıtlarda karşılaştırıcıyla seçme (Örnek 9).

4.2 Kabarcık sıralaması (bubble sort)

1 · Problem. Aynı. **2 · Fikir.** Dizi soldan sağa taranır; **komşu** iki öğe ters sıradaysa takas edilir. Bir geçişin sonunda kalanların en büyüğü, sudaki bir kabarcık gibi sona "yüzer" ve yerine oturur; sonraki geçiş bir

öğe kısa yapılır. Bir geçişte hiç takas olmazsa bütün komşu çiftler sıralıdır, yani dizi sıralıdır: **erken çıkış bayrağı** bu durumda döngüyü bitirir.

Şekil 5.9
Kabarcık sıralaması, a = {42, 17, 8, 31, 25, 4, 39, 12}: her geçişten sonra dizi (yeşil: yerine oturan sonek; sarı: bu geçişte yeri değişen öğeler) ve o ana kadarki sayaçlar



3 · Elle izleme. İlk geçişte 42 her komşusundan büyük olduğu için yedi takasla sona taşınır (Şekil 5.9). Dizi 5. geçişte sıralanır, ama algoritma bunu ancak 6. geçişte hiç takas görmeyince anlar: toplam 6 geçiş, 27 karşılaştırma, 17 takas. Takas sayısı ters çevrim sayısına (17) eşittir: her komşu takası tam bir ters çevrimi giderir.

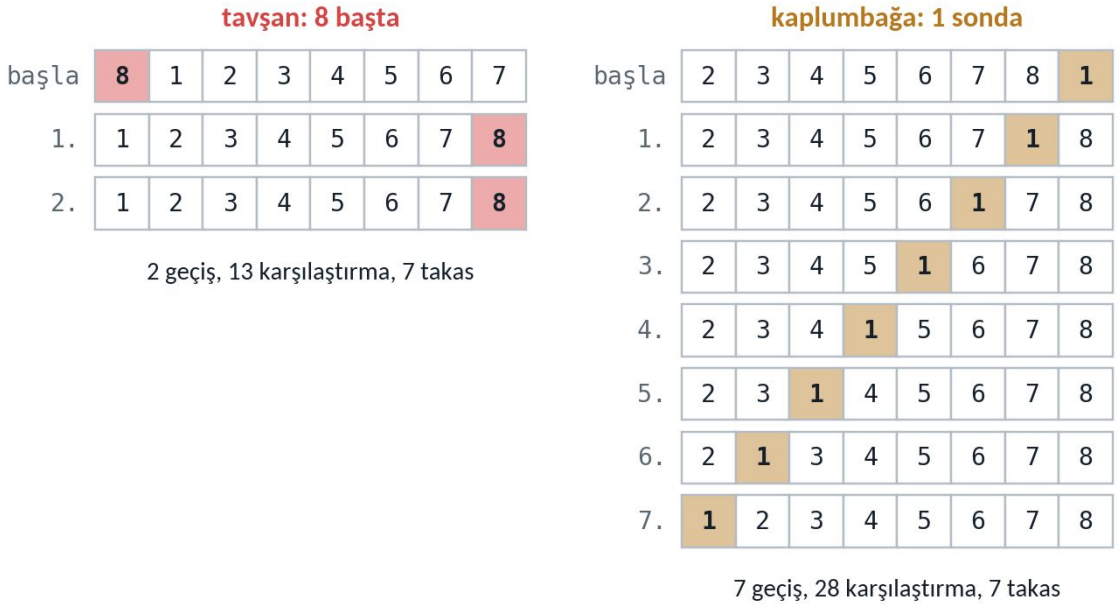
> TAHMİN ET · **Bir sonraki takas hangisi?**
>
> 2. geçişte a[0] ↔ a[1] takası yapıldı; dizi şimdi {8, 17, 31, 25, 4, 39, 12, 42} ve j = 1. Sıradaki takas hangi iki indis arasında olur? (Yanıt: a[1] = 17 ≤ a[2] = 31 takas gerektirmez; sıradaki takas a[2] ↔ a[3], yani 31 ↔ 25 olur.)

Kabarcık sıralamasının asimetrik bir zaafı vardır (Şekil 5.10). Başta duran büyük bir öğe (“tavşan”) tek geçişte sona gider; sonda duran küçük bir öğe (“kaplumbağa”) ise her geçişte yalnızca bir adım sola ilerler. Örnek 2'nin tablosu bunu sayar: tavşanlı girdi 2 geçiş ve 13 karşılaştırmada, kaplumbağalı girdi ise yalnızca 7 ters çevrime karşın 7 geçiş ve 28 karşılaştırmada biter. (Tavşan ve kaplumbağa adları yaygın, betimleyici bir adlandırmadır.)

SÖZDE KOD · 4 · Sözde kod: erken çıkışlı kabarcık sıralaması

```
algoritma KABARCIK-SIRALA(a[0..n−1])
    i ← 0'dan n−2'ye kadar her i için
        takas_oldu ← yanlış
        j ← 0'dan n−2−i'ye kadar her j için
            eğer a[j] > a[j+1] ise
                a[j] ile a[j+1]'i takas et; takas_oldu ← doğru
        eğer takas_oldu değil ise dur      ▷ erken çıkış: dizi sıralı
```

Şekil 5.10
Tavşan ve kaplumbağa: başta duran büyük öğe tek geçişte sona gider; sonda duran küçük öğe geçiş başına bir adım ilerler (Örnek 2)



5 · Doğruluk. Değişmez (Tablo 5.3): i geçişten sonra son i konum (a[n−i..n−1]) dizinin en büyük i elemanını sıralı olarak içerir. **Koruma:** i. geçiş a[0..n − 1 − i] aralığını tarar; tarama boyunca o ana kadar görülen en büyük öğe her adımda bir sağa taşınır, geçiş sonunda a[n − 1 − i]'ye oturur. **Sonlanma:** ya n − 1 geçiş tamamlanır (son i + 1 konum yerinde, kalan tek öğe en küçüktür) ya da bir geçişte takas olmaz; o zaman her komşu çift a[j] ≤ a[j + 1]'dir ve zincirleme olarak bütün dizi sıralıdır. Kararlılık: takas yalnızca **kesin** büyüklükte (>) yapılır; eşit komşular yer değiştirmez.

6 · Analiz. p. geçiş n − p karşılaştırma yapar. En iyi durumda (sıralı girdi) ilk geçişte takas olmaz: n − 1 karşılaştırma, O(n). En kötü durumda (ters sıralı) n − 1 geçişin hepsi çalışır: n(n − 1)/2 karşılaştırma ve n(n − 1)/2 takas. Takas sayısı her girdide ters çevrim sayısına eşittir; rastgele girdide ortalama n(n − 1)/4 takas ve $n^2/2$'ye yakın karşılaştırma yapılır, yani ortalama durum da $\Theta(n^2)$'dir. Uyarlanabilirliği **zayıftır**: sıralı veride hızlıdır, ama tek bir kaplumbağa n − 1 geçiş ister. Ek bellek O(1).

Tablo 5.3
Kabarcık sıralaması (erken çıkışlı) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Kabarcık sıralaması (erken çıkışlı)** *(bubble sort (with early exit))*
Strateji	Artımlı (her geçişte kalanların en büyüğü sona "yüzer"): komşu karşılaştırma ve takas; geçişte takas yoksa dur
Girdi önkoşulu	Yok — karşılaştırılabilir anahtarlar
Zaman (en iyi · ortalama · en kötü)	O(n) · $\Theta(n^2)$ · $\Theta(n^2)$
Ek bellek	O(1)
Özellikler	Kararlı (eşitler takas edilmez) · yerinde · uyarlanabilir (zayıf) · takas sayısı = ters çevrim sayısı. Sıralı girdide n − 1 karşılaştırma; tek bir "kaplumbağa" (sonda küçük eleman) n − 1 geçiş ister (n = 8: 7 geçiş, 28 karşılaştırma)
Değişmez / doğruluk	i geçişten sonra son i konum (a[n−i..n−1]) dizinin en büyük i elemanını sıralı olarak içerir
Kütüphane karşılığı	C: yok (elle) · C++: yok · Python: yok

Ne zaman kullanılır	Öğretimde; çok küçük ya da hemen hemen sıralı diziler için “sıralı mı?” denetimiyle birlikte
Ne zaman kullanılmaz	Genel amaçlı sıralama: rastgele veride araya eklemeden yavaştır (bu makinede n = 10 000, C -O2: ≈ 7,3 kat; Astrachan (2003) Java'da dizgilerle “neredeyse üç kat” bildirir)

7 · C gerçekleştirimi. (Örnek 2; bayrak her geçişin başında sıfırlanır, Hata Avcısı 11):

c/H05_02_kabarcik_siralama.c · satır 37–61

```c
static void kabarcik_sirala(int *a, size_t n, int yazdir)
{
    for (size_t i = 0; i + 1 < n; i++) {
        int takas_oldu = 0;              /* bu geçişte takas var mı? */
        long long onceki = takas_sayisi;
        for (size_t j = 0; j + 1 < n - i; j++) {
            karsilastirma++;
            if (a[j] > a[j + 1]) {       /* komşular ters sırada */
                takas(&a[j], &a[j + 1]);
                takas_sayisi++;
                takas_oldu = 1;
            }
        }
        gecis_sayisi++;
        if (yazdir) {
            printf("geçiş %lld: karşılaştırma = %zu, takas = %lld\n",
                   gecis_sayisi, n - 1 - i, takas_sayisi - onceki);
            printf("      ");
            dizi_yaz(a, n, n - 1 - i);
        }
        if (!takas_oldu) {
            break;                       /* erken çıkış: dizi sıralı */
        }
    }
}
```

İç döngü koşulu `j + 1 < n - i` erişilen en büyük indisin (`j + 1`) sınırıdır; `j < n - i` yazmak a[n]'i okur (Hata Avcısı 10). Örnek 2'nin ikinci bölümü erken çıkışın beş girdideki etkisini sayar (geçiş, karşılaştırma, takas):

ÇIKTI · ÖRNEK 2 (İKİNCİ BÖLÜM)

```
$ ./H05_02_kabarcik_siralama
geçiş  karş.  takas   girdi
    1      7      0   sıralı
    2     13      2   neredeyse sıralı (iki komşu takası)
    2     13      7   tavşan: 8 başta (bir geçişte sona gider)
    7     28      7   kaplumbağa: 1 sonda (geçiş başına 1 adım)
    7     28     28   ters sıralı (en kötü durum)
```

8 · Üç dil karşılaştırması. Bayrak C'de `int` (0/1), C++ ve Python'da `bool`'dur (`bool takas_oldu = false;`, `takas_oldu = False`). C++'ta komşu takası `std::swap(a[j], a[j + 1])`, Python'da `a[j], a[j + 1] = a[j + 1], a[j]` (satır 33); Python'da iç döngü `range(n - 1 - i)` ile yazılır, üst sınır dahil değildir. Tam fonksiyonlar Ek A.2'dedir.

9 · Test. Örnek 6 kâhin testinde kabarcık sıralaması 628/628 dizide sıralı, 628/628 dizide kâhinle aynı çıktı. **10 · Deney.** Rastgele girdide kabarcık sıralaması üç algoritmanın en yavaşıdır: n = 10 000'de C'de (-O2) araya eklemeden 7,3 kat yavaş ölçüldü (Python'da n = 2 000'de 2,2 kat). Takas sayısı araya eklemenin kaydırma sayısıyla aynıdır; fark, bir takasın üç atama (geçici değişkenle) olması ve karşılaştırma sayısının $n^2/4$ yerine $n^2/2$'ye yakın olmasıdır. Astrachan'ın (2003) Java'da dizgilerle bildirdiği "neredeyse üç kat"tan farklı bu oran, sabitin dile ve veri türüne bağlı olduğunu gösterir. **11 · Varyantlar.** Çift yönlü kabarcık (kokteyl) sıralaması: geçişleri bir soldan sağa, bir sağdan sola yaparak kaplumbağaları da hızlı taşır (Alıştırma 5.12); son takasın konumunu hatırlayıp sonraki geçişi orada bitirmek.

4.3 Araya ekleme sıralaması (insertion sort)

1 · Problem. Aynı. **2 · Fikir.** İskambil kâğıtlarını sıralarken yaptığımız gibi: soldaki kâğıtlar sıralı bir eldir; sıradaki kâğıt (**anahtar**) ele alınır, ondan büyük kâğıtlar birer sağa kaydırılır ve anahtar açılan **boşluğa** konur. Seçmeli sıralama doğru öğeyi **seçip** doğru yere götürür; araya ekleme sıradaki öğeyi alıp sıralı önekte **doğru yeri bulur**.

3 · Elle izleme. Şekil 5.11 her adımı gösterir: sıralı önek her satırda bir öğe büyür. i = 1 … 7 için kaydırma sayıları sırasıyla 1, 2, 1, 2, 5, 1, 5'tir; toplam 17, yani yine ters çevrim sayısı. Karşılaştırma sayısı 21'dir: her anahtar, kaydırdığı her öğe için bir ve kendisini durduran öğe için bir karşılaştırma yapar; başa (a[0] konumuna) yerleşen anahtarlar durduran öğe bulamaz.

> TAHMİN ET · **Sizce bu anahtar kaç kaydırma yapar?**
>
> i = 5'te dizi {8, 17, 25, 31, 42, 4, 39, 12} ve anahtar a[5] = 4. Kaç öğe kaydırılır, anahtar hangi indise yerleşir? (Yanıt: soldaki 5 öğenin hepsi 4'ten büyük olduğundan 5 kaydırma yapılır ve anahtar a[0]'a yerleşir; j = −1'e inildiği için bu anahtarı durduran bir karşılaştırma olmaz.)

Şekil 5.11
Araya ekleme sıralaması, a = {42, 17, 8, 31, 25, 4, 39, 12}: her adımdan sonra dizi (yeşil: sıralı önek; sarı: anahtarın yeni yeri) ve o ana kadarki sayaçlar



SÖZDE KOD · 4 · Sözde kod: araya ekleme sıralaması

```
algoritma ARAYA-EKLEME-SIRALA(a[0..n−1])
    i ← 1'den n−1'e kadar her i için
        anahtar ← a[i];  j ← i − 1
        j ≥ 0 ve a[j] > anahtar iken          ▷ önce sınır, sonra öğe
            a[j+1] ← a[j];  j ← j − 1          ▷ bir sağa kaydır
        a[j+1] ← anahtar                       ▷ boşluğa koy
```

5 · Doğruluk. Değişmez (Tablo 5.4): i. adımın başında a[0..i−1], girdinin ilk i elemanının sıralı hâlidir. **Başlatma:** i = 1'de önek tek öğedir, sıralıdır. **Koruma:** iç döngü boyunca boşluk a[j + 1]'dedir ve a[j + 2..i] aralığındaki öğelerin hepsi anahtardan büyüktür; döngü a[j] ≤ anahtar ya da j = −1 olunca durur ve anahtar boşluğa konduğunda a[0..i] sıralı olur. **Sonlanma:** i = n'de a[0..n − 1] girdinin sıralı hâlidir. Seçmeli sıralamadan farkına dikkat edin: araya eklemede önek "ilk i öğenin" sıralı hâlidir, **dizinin en küçükleri** değildir (Şekil 5.13). Döngü koşulu `a[j] > anahtar` olduğu için eşit bir öğenin önünde durulur: algoritma kararlıdır.

6 · Analiz. T ters çevrim sayısı olsun. Kaydırma sayısı tam T'dir; karşılaştırma sayısı T + (n − 1) − z'dir (z: başa yerleşen, yani durduran karşılaştırması olmayan anahtar sayısı; Örnek 4 bu eşitliği denetler). Sıralı girdide T = 0: n − 1 karşılaştırma, O(n). Ters sıralı girdide T = n(n − 1)/2: $\Theta(n^2)$. Rastgele girdide T ≈ n(n − 1)/4: yaklaşık $n^2/4$ karşılaştırma ve kaydırma, yani seçmeli ve kabarcık sıralamasının yaklaşık yarısı kadar iş; ama yine $\Theta(n^2)$. Süre Θ(n + T)'dir: algoritma **uyarlanabilir**dir. Her öğe sıralı yerinden en çok k uzaktaysa T ≤ n·k olur ve süre O(nk)'dir (Örnek 11, Bölüm 5.3). Algoritma **çevrimiçidir**: yeni gelen öğe aynı `yerlestir` adımıyla sıralı diziye eklenebilir.

Tablo 5.4
Araya ekleme sıralaması kimlik kartı

ALGORİTMA KİMLİK KARTI	**Araya ekleme sıralaması** *(insertion sort)*
Strateji	Artımlı (sıralı önek): sıradaki anahtarı, ondan büyükleri bir sağa kaydırarak yerine koy
Girdi önkoşulu	Yok — karşılaştırılabilir anahtarlar
Zaman (en iyi · ortalama · en kötü)	O(n) · $\Theta(n^2)$ · $\Theta(n^2)$
Ek bellek	O(1)
Özellikler	Kararlı · yerinde · uyarlanabilir · çevrimiçi; kaydırma sayısı = ters çevrim sayısı, süre ∝ n + ters çevrim; k-sıralı dizide O(nk) (n = 1000, k = 8: 1 994 kaydırma)
Değişmez / doğruluk	i. adımın başında a[0..i−1], girdinin ilk i elemanının sıralı hâlidir
Kütüphane karşılığı	C: yok (elle); kaydırma için `memmove` · C++: yok; sıralı vektöre ekleme `std::upper_bound` + `insert` · Python: yok; sıralı listeye ekleme `bisect.insort`
Ne zaman kullanılır	Küçük diziler (kütüphaneler küçük alt dizilerde kullanır: introsort, Timsort); neredeyse sıralı veri; veri parça parça geldiğinde (çevrimiçi)
Ne zaman kullanılmaz	Büyük ve rastgele veri ($\Theta(n^2)$): birleştirmeli ya da hızlı sıralama (H6)

7 · C gerçekleştirimi. (Örnek 3; tek bir anahtarın yerleştirilmesi ayrı bir fonksiyondur, böylece çevrimiçi ekleme aynı kodu kullanır):

C c/H05_03_araya_ekleme.c · satır 36–47

```c
static size_t yerlestir(int *a, size_t i)
{
    int anahtar = a[i];
    ptrdiff_t j = (ptrdiff_t)i - 1;   /* -1 olabilir */
    while (j >= 0 && buyuk_mu(a[j], anahtar)) {
        a[j + 1] = a[j];   /* bir sağa kaydır */
        kaydirma++;
        j--;
    }
    a[j + 1] = anahtar;
    return (size_t)(j + 1);
}
```

İç döngünün sayacı `j` −1'e inebildiği için işaretli `ptrdiff_t` türündedir; `size_t` ile `j >= 0` her zaman doğru olurdu. Koşuldaki sıra önemlidir: `&&` kısa devre yapar, `j >= 0` yanlışsa `a[j]` hiç okunmaz (Hata Avcısı 2). Örnek 3'ün ikinci bölümü sıralı diziye sonradan gelen öğeleri ekler:

ÇIKTI · ÖRNEK 3 (ÇEVRİMİÇİ EKLEME)

```
$ ./H05_03_araya_ekleme
Çevrimiçi: yeni gelen eleman sıralı diziye eklenir
+15 -> yeri a[3], karşılaştırma = 6, kaydırma = 5
+50 -> yeri a[9], karşılaştırma = 1, kaydırma = 0
+ 1 -> yeri a[0], karşılaştırma = 10, kaydırma = 10
```

8 · Üç dil karşılaştırması. C++'ta `auto j = static_cast<std::ptrdiff_t>(i) - 1` aynı işaretli türü verir; Python tamsayısının işareti zaten vardır. `&&` ile `and` aynı kısa devre kuralına uyar:

C++ H05_03_araya_ekleme.cpp · satır 40–42

```cpp
    int anahtar = a[i];
    auto j = static_cast<std::ptrdiff_t>(i) - 1;
    while (j >= 0 && buyuk_mu(a[j], anahtar)) {
```

Python H05_03_araya_ekleme.py · satır 33–37

```python
    j = i - 1
    while j >= 0 and buyuk_mu(a[j], anahtar):
        a[j + 1] = a[j]    # bir sağa kaydır
        kaydirma += 1
        j -= 1
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> İç döngüde j −1'e inebildiği için C'de işaretli `ptrdiff_t` kullanılır; C++'ta `auto j = static_cast<std::ptrdiff_t>(i) - 1` aynı türü verir (`auto`: tür sağ taraftan çıkarılır). `size_t` ile `j >= 0` her zaman doğru olurdu. `&&` / `and` kısa devre yapar: `j >= 0` yanlışsa `a[j]` hiç okunmaz (Hata Avcısı 2). Çevrimiçi eklemede C sabit kapasiteli diziye `a[n] = x` yazar; C++'ta `push_back`, Python'da `append` diziyi büyütür.

9 · Test. Örnek 6'da araya ekleme 628/628 dizide sıralı, 628/628 dizide kâhinle aynı; Örnek 4 sayaçları kapalı biçimlerle karşılaştırır (Bölüm 4.5). **10 · Deney.** n = 10 000'de araya ekleme rastgele girdide sıralı girdiden 1 696 kat yavaştır (C, -O2): sıralıda 9 999, neredeyse sıralıda 34 872, rastgelede 24 726 789 karşılaştırma. **11 · Varyantlar.** İkili aramalı araya ekleme: yeri H4'teki üst sınır (upper_bound) aramasıyla bul, karşılaştırma O(n log n)'e iner, kaydırma yine T kalır (Alıştırma 5.13); kaydırmayı `memmove` ile blok yapmak (Bölüm 3.8); uzak öğeleri önce kaba aralıklarla yaklaştırmak: Shell sıralaması (Bölüm 4.4).

4.4 Merak: Shell sıralaması (Shellsort)

1 · Problem ve fikir. Araya ekleme öğeleri yalnızca birer konum kaydırır: uzaktaki bir öğe birçok adım ister. Shell sıralaması önce birbirinden h uzaklıktaki öğelerin alt dizilerini araya eklemeyle sıralar (**h-sıralama**), sonra h'yi küçültür; son geçiş (h = 1) sıradan araya eklemedir, ama dizi artık neredeyse sıralıdır. **3 · Elle izleme.** Ortak dizide h = 4, 2, 1 (Şekil 5.12).

Şekil 5.12
Shell sıralaması, h = 4, 2, 1: aynı renkli kutular aynı alt dizidir; her geçişten sonra dizi h-sıralıdır (Örnek 13)



4 · Sözde kod ve C. Her h için `h_gecisi` uygulanır (Örnek 13); bu, kaydırma adımı h olan araya eklemedir. Boşluk indisi `j` işaretsizdir, `j >= h` koşulu `j - h`'nin negatife düşmesini önler:

c/H05_13_shell_siralama.c · satır 33–49

```c
static void h_gecisi(int *a, size_t n, size_t h)
{
    for (size_t i = h; i < n; i++) {
        int anahtar = a[i];
        size_t j = i;                       /* boşluğun konumu */
        while (j >= h) {
            karsilastirma++;
            if (a[j - h] <= anahtar) {
                break;
            }
            a[j] = a[j - h];   /* h sağa kaydır */
            kaydirma++;
            j -= h;
        }
```

```
47          a[j] = anahtar;
48      }
49  }
```

5 · Doğruluk. h-geçişinden sonra dizi h-sıralıdır (her i ≥ h için a[i − h] ≤ a[i]); son geçiş (h = 1) araya eklemenin kendisidir, sonuç sıralıdır. **6 · Analiz.** Çözümlemesi aralık dizisine bağlıdır ve bu haftanın kapsamını aşar. Shell'in özgün aralıkları (⌊n/2⌋, ⌊n/4⌋, …, 1; Örnek 13'teki “Shell (n/2)”) en kötü durumda Θ(n²) karşılaştırma yapar: n ikinin kuvvetiyken tek ve çift konumlardaki öğeler son geçişe (h = 1) kadar birbiriyle karşılaştırılmaz (Weiss, 1997). Knuth'un 1, 4, 13, 40, … (h ← 3h + 1) aralıklarıyla en kötü durum O(n^(3/2))'dir (Knuth, 1998). Ciura (2001), ortalama durumda iyi sonuç veren 1, 4, 10, 23, 57, 132, 301, 701 aralıklarını deneysel aramayla bulmuştur; en iyi aralık dizisi hâlâ bilinmemektedir. Örnek 13 üç aralık dizisini ve sıradan araya eklemeyi n = 1000 ve 5 000 rastgele permütasyonda sayar.

Kimlik kartı (özet). En iyi O(n log n), ortalama: aralığa bağlı, en kötü O(n√n) (Knuth aralıkları); ek bellek O(1); kararsız ve yerinde (Tablo 5.1). Kartın tamamı web sayfasındaki algoritma kataloğundadır. **8 · Üç dil.** `h_gecisi`'nin C++ ve Python karşılıkları Ek A.13'tedir. **9 · Test.** Program her sonucu “sıralı mı?” diye denetler ve her geçişten sonra “h-sıralı mı?” sorusunu yanıtlar. **10 · Deney.** n = 5 000'de karşılaştırma sayıları: Ciura aralıkları 87 458, Knuth aralıkları 101 009, araya ekleme 6 224 400. **11 · Varyantlar ve sınır.** Shell sıralaması kararsızdır: uzak kaydırmalar eşit anahtarları birbirinin üstünden atlatır (Örnek 13'ün son bölümü: `sonuç : 1b 1a 1c 2a 2c 2b`).

4.5 Üç algoritmanın karşılaştırması

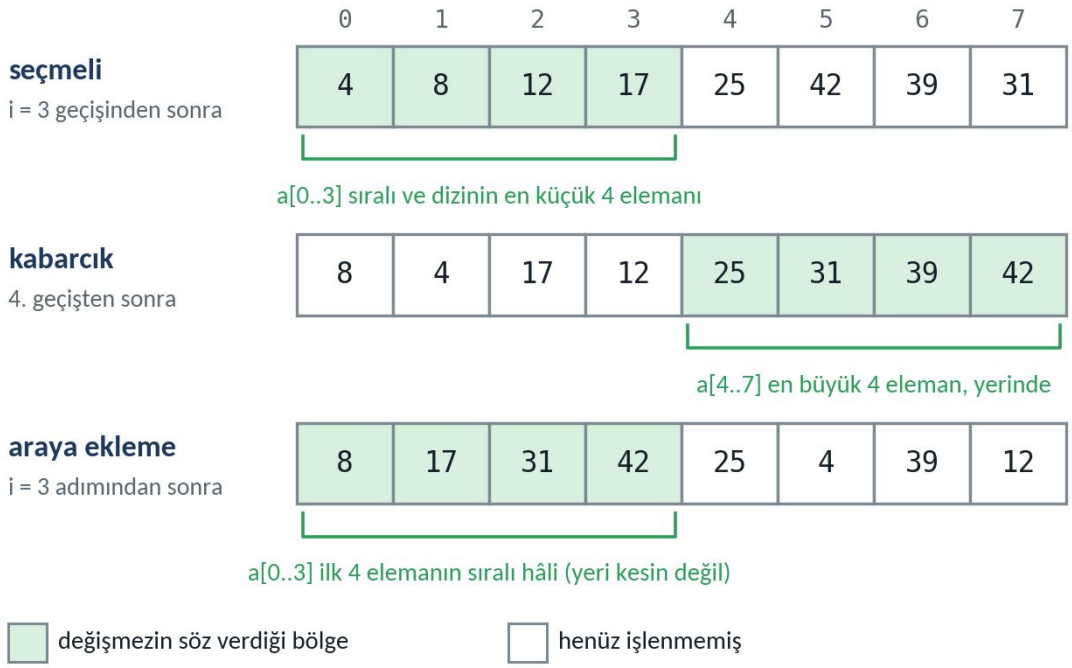
Tablo 5.5 üç algoritmanın sayaçlarını kapalı biçimde verir (T: ters çevrim sayısı). Ortak dizide (n = 8, T = 17) seçmeli 28 karşılaştırma ve 5 takas, kabarcık 27 karşılaştırma ve 17 takas, araya ekleme 21 karşılaştırma ve 17 kaydırma yapar: seçmeli en çok karşılaştırma ve en az takas yapar; kabarcık ile araya ekleme aynı sayıda eleman hareketi (ters çevrim kadar) yapar, ama araya ekleme daha az karşılaştırmayla (Şekil 5.14).

Tablo 5.5
Sayaçların kapalı biçimleri (T: ters çevrim sayısı; z: başa yerleşen anahtar sayısı)

Algoritma	Karşılaştırma: en iyi	en kötü	ortalama (rastgele)	Takas / kaydırma
Seçmeli	n(n − 1)/2	n(n − 1)/2	n(n − 1)/2	en çok n − 1 takas (ters sıralıda ⌊n/2⌋)
Kabarcık (erken çıkışlı)	n − 1	n(n − 1)/2	≈ n²/2	takas = T
Araya ekleme	n − 1	n(n − 1)/2	≈ n²/4	kaydırma = T; karşılaştırma = T + (n − 1) − z

Üç algoritmanın değişmezleri de farklı bölgelere söz verir (Şekil 5.13): seçmeli sıralamanın öneki hem sıralı hem de dizinin en küçükleridir; kabarcık sıralamasının soneki en büyüklerdir ve yerine oturmuştur; araya eklemenin öneki ise yalnızca ilk i öğenin sıralı hâlidir, sonraki bir anahtar araya girebilir.

Şekil 5.13
Değişmez şeritleri, dört öğe işlendikten sonra: üç algoritmanın değişmezi farklı bölgeye ve farklı söze dayanır



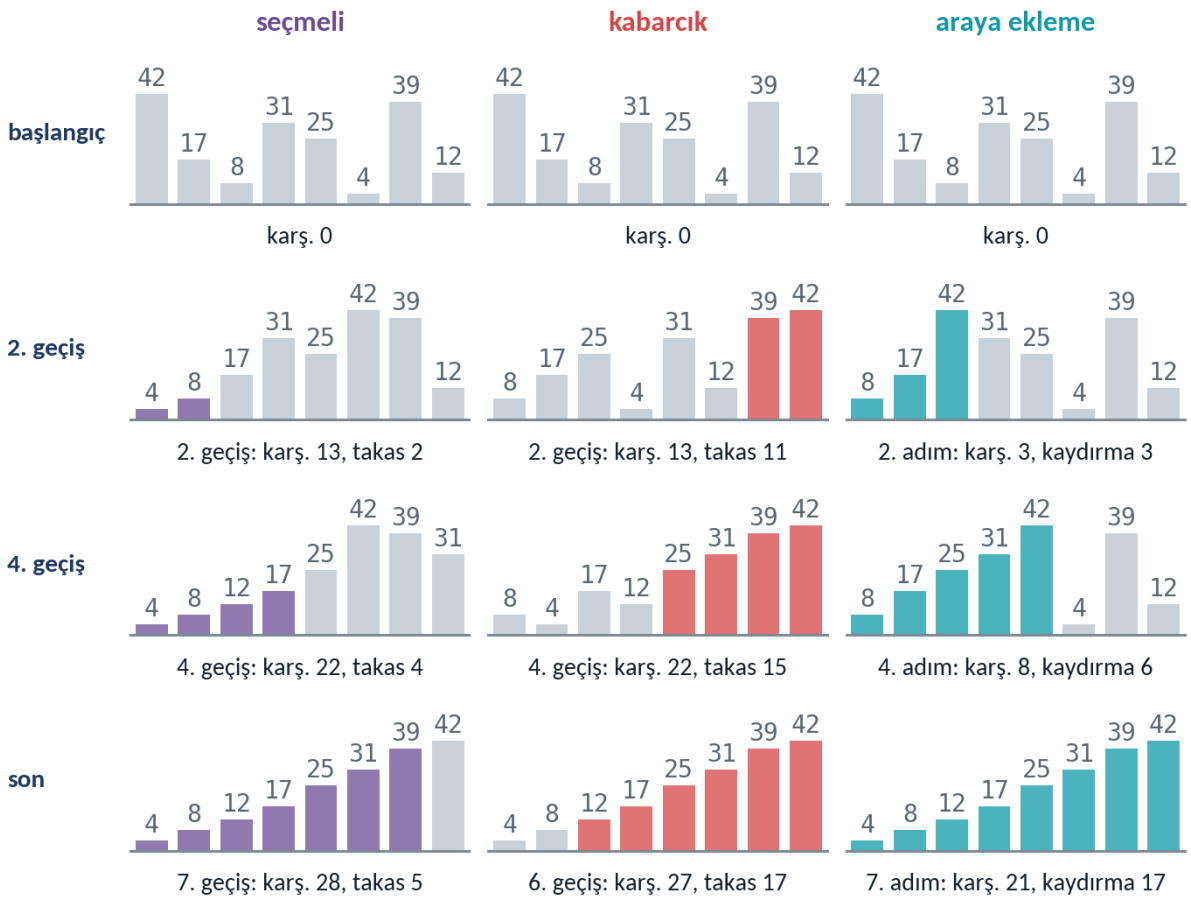
Örnek 4 üç algoritmayı n = 100 ve 1000 için üç girdi türünde sayar ve rastgele girdide kapalı biçimleri programla denetler (z'yi de program hesaplar):

ÇIKTI · ÖRNEK 4 (N = 1000)

```
$ ./H05_04_sayac_karsilastirma
n = 1000   n(n-1)/2 = 499500   n(n-1)/4 = 249750.00
algoritma     girdi         karşılaştırma  takas  kaydırma
seçmeli       rastgele             499500    993         0
seçmeli       sıralı               499500      0         0
seçmeli       ters sıralı          499500    500         0
kabarcık      rastgele             498905 255893         0
kabarcık      sıralı                  999      0         0
kabarcık      ters sıralı          499500 499500         0
araya ekleme  rastgele             256883      0    255893
araya ekleme  sıralı                  999      0         0
araya ekleme  ters sıralı          499500      0    499500
kuram denetimi (rastgele girdi):
  ters çevrim = 255893  (beklenen değer n(n-1)/4 = 249750.00)
  kabarcık takas = ters çevrim: evet
  araya ekleme kaydırma = ters çevrim: evet
  araya ekleme karşılaştırma = kaydırma + (n-1) - z
    = 255893 + 999 - 9 = 256883: evet
```

Tablodaki her satır bir analiz cümlesinin sınamasıdır: seçmelinin üç girdide aynı karşılaştırma sayısı, ters sıralıda yalnızca 500 takas (koşullu takas, $\lfloor n/2 \rfloor$), kabarcığın takası ile araya eklemenin kaydırmasının ters çevrime eşitliği, ikisinin sıralı girdide $n - 1$ karşılaştırması. Türkçe etiketlerin hizalanması için program UTF-8 karakterlerini sayar (`printf("%-14s")` bayt sayar; Dil Penceresi).

Şekil 5.14
Üç algoritma aynı dizide (a = {42, 17, 8, 31, 25, 4, 39, 12}) aynı anlarda: çubuk yüksekliği değerdir, renkli çubuklar değişmezin bölgesidir; altta o ana kadarki sayaçlar



5. Çözümlü Örnekler

Örnek merdiveninin on üç basamağı Tablo 5.6 içinde ders notundaki yerleriyle listelenmiştir; her basamak C, C++ ve Python'da yazılmıştır ve üç dilin çıktısı bayt bayt aynıdır (`esdeger_test.py`). Bölüm 3 ve 4'te ele alınmayan üç basamak (Örnek 6, 8 ve 11) aşağıda C koduyla ve gerçek çıktısıyla çözülür; C++ ve Python karşılıkları Ek A'dadır.

Tablo 5.6
Örnek merdiveni ve ders notundaki yerleri (= : üç dilde aynı çıktı)

#	Örnek	Yeni öge / amaç	Düzey	Çıktı	Bölüm
1	Seçmeli sıralama ve her geçişte dizinin yazdırılması	Değişmez: önek sıralı ve en küçükler	★	=	4.1
2	Kabarcık sıralaması (erken çıkış bayraklı)	Komşu takası	★	=	4.2
3	Araya ekleme sıralaması	Kaydırarak yerleştirme	★	=	4.3
4	Karşılaştırma/takas sayaçlarıyla üç algoritma: rastgele, sıralı, ters sıralı girdi	Durum analizi	★★	=	4.5
5	Ters çevrim sayısı ($O(n^2)$) ve araya ekleme kaydırma sayısıyla eşitliği	Ters çevrim kavramı	★★	=	3.3
6	"Dizi sıralı mı?" ve kütüphane sıralamasıyla kâhin testi	Karşılaştırmalı test	★★	=	5.1

#	Örnek	Yeni öge / amaç	Düzey	Çıktı	Bölüm
7	qsort ile int, double ve string dizisi sıralama; taşmaya güvenli karşılaştırıcı	Genel sıralama arayüzü	★★	=	3.5
8	Yapı dizisini ortalamaya göre azalan sıralama	Yapı karşılaştırıcısı	★★	=	5.2
9	Kararlılık deneyi: eşit ortalamalı kayıtlarda seçmeli ve araya ekleme	Kararlılık	★★	=	3.4
10	Çok anahtarlı sıralama: (ortalama ↓, soyad ↑, ad ↑) tek karşılaştırıcıyla ve kararlı ardışık geçişlerle	Sözlük sırası ve kararlılık	★★★	=	3.6
11	k-sıralı (neredeyse sıralı) dizide araya eklemenin O(nk) davranışı	Uyarlanabilirlik	★★★	=	5.3
12	İşaretçi dizisi sıralama: büyük kayıtları taşımadan sıralama	Dolaylı sıralama	★★★	=	3.7
13	Shell sıralaması ve aralık dizisi deneyi (merak)	h-sıralama	★★★	=	4.4

5.1 Örnek 6: “sıralı mı?” ve kâhin testi

Bir sıralama fonksiyonunu nasıl sınarız? İlk akla gelen, çıktının sıralı olup olmadığını denetlemektir (n − 1 karşılaştırma). Örnek 6 bunun yetmediğini kasıtlı olarak hatalı bir “sahte” sıralamayla gösterir: `sahte_sirala` her öğeyi önekteki en büyükle değiştirir; sonuç her zaman sıralıdır, ama girdinin permütasyonu değildir. **Kâhin** (test oracle) testi çıktıyı güvendiğimiz bir başvuru uygulamasının, burada kütüphane sıralamasının çıktısıyla karşılaştırır ve bu hatayı yakalar.

C **c/H05_06_kahin_testi.c · satır 75–82**

```c
75  static void sahte_sirala(int *a, size_t n)
76  {
77      for (size_t i = 1; i < n; i++) {
78          if (a[i] < a[i - 1]) {
79              a[i] = a[i - 1];
80          }
81      }
82  }
```

C **c/H05_06_kahin_testi.c · satır 103–108**

```c
103 /* Kâhin: kütüphane sıralamasının sonucu (yalnızca test için) */
104 static void kahin(const int *girdi, size_t n, int *beklenen)
105 {
106     memcpy(beklenen, girdi, n * sizeof girdi[0]);
107     qsort(beklenen, n, sizeof beklenen[0], int_karsilastir);
108 }
```

ÇIKTI · ÖRNEK 6

```
$ ./H05_06_kahin_testi
test kümesi: 8 özel dizi + n = 0..30 için 20'şer rastgele dizi = 628 dizi
algoritma          sıralı mı  kâhinle aynı
seçmeli              628/628        628/628
kabarcık             628/628        628/628
araya ekleme         628/628        628/628
sahte (hatalı)       628/628         62/628
kâhinin yakaladığı ilk hata:
  girdi  [2 1]
  çıktı  [2 2]  sıralı mı: evet
```

```
  kâhin  [1 2]
```

Test kümesi iki parçadır: sınır durumları (boş dizi, tek öğe, iki öğe, hepsi eşit, ters sıralı, sıralı, `INT_MIN`/`INT_MAX` içeren dizi) ve n = 0 … 30 için 20'şer rastgele dizi (yarısı çok yinelenen 0..9 değerleriyle). Kâhinin dili önemli değildir: `int` dizisinde eşit öğeler ayırt edilemediği için kararlı ya da kararsız her doğru sıralama aynı sonucu verir. **Karşılaştırmalı test** (differential testing) diye de anılan bu yöntemi dönem boyunca her algoritmada kullanacağız.

5.2 Örnek 8: yapı dizisini karşılaştırıcıyla sıralamak

Örnek 8 on öğrenci kaydını (`no, ad, soyad, ortalama`) kendi araya ekleme sıralamamızla ortalamaya göre **azalan** sıraya dizer. Algoritma kaydın içini bilmez; karşılaştırıcıyı bir fonksiyon işaretçisi olarak alır. Azalan sıra için karşılaştırmanın işaretleri ters çevrilir. Döngü `kars(&a[j], &anahtar) <= 0` olunca, yani eşitte de durur; bu yüzden eşit ortalamalılar kayıt sırasını korur:

c/H05_08_yapi_siralama.c · satır 25–44

```c
25 static int ortalama_azalan(const Ogrenci *x, const Ogrenci *y)
26 {
27     return (x->ortalama < y->ortalama) - (x->ortalama > y->ortalama);
28 }
29
30 /* Araya ekleme: kars(a[j], anahtar) > 0 oldukça kaydır (kararlı) */
31 static void ogrenci_sirala(Ogrenci *a, size_t n, ogrenci_kars kars)
32 {
33     for (size_t i = 1; i < n; i++) {
34         Ogrenci anahtar = a[i];             /* yapı ataması: kopya */
35         ptrdiff_t j = (ptrdiff_t)i - 1;
36         while (j >= 0) {
37             karsilastirma++;
38             if (kars(&a[j], &anahtar) <= 0) {
39                 break;
40             }
41             a[j + 1] = a[j];
42             j--;
43         }
44         a[j + 1] = anahtar;
```

ÇIKTI · ÖRNEK 8 (SIRALI TABLONUN BAŞI)

```
$ ./H05_08_yapi_siralama
Ortalamaya göre azalan (araya ekleme, karşılaştırıcıyla):
sıra  ortalama   no  ad soyad
  1.     90.00  104  Can Yılmaz
  2.     90.00  108  Deniz Arslan
  3.     78.50  101  Ayşe Yılmaz
  4.     78.50  103  Zeynep Demir
  5.     78.50  105  Elif Kaya
  6.     78.50  107  Ali Yılmaz
  …
karşılaştırma = 24
eşit ortalamalılarda kayıt sırası (no artan) korundu mu? evet
```

Ortalamalar 90,00 / 78,50 / 65,25 gibi ikili tabanda tam gösterilebilen değerlerden seçilmiştir; kayan nokta eşitliği burada güvenilirdir. Gerçek notlarda (ör. 0,4·vize + 0,6·final) matematikte eşit iki ortalama son bitte farklı hesaplanabilir (0,4·3 + 0,6·0 = 1,2000000000000002, ama 0,4·0 + 0,6·2 = 1,2); Mini-OBS 2'nin başarı ölçütü bu yüzden ağırlıklı ortalamayı ×100 tamsayı olarak karşılaştırır (Bölüm 10). C'de `Ogrenci anahtar = a[i]` yapının tamamını kopyalar; Python'da aynı yazım yalnızca nesne başvurusunu kopyalar (Ek A.8). Aynı on kayıt Örnek 9 ve 10'da kararlılık ve çok anahtarlı sıralama deneylerinde kullanılır (Bölüm 3.4 ve 3.6).

5.3 Örnek 11: k-sıralı dizide araya ekleme, O(nk)

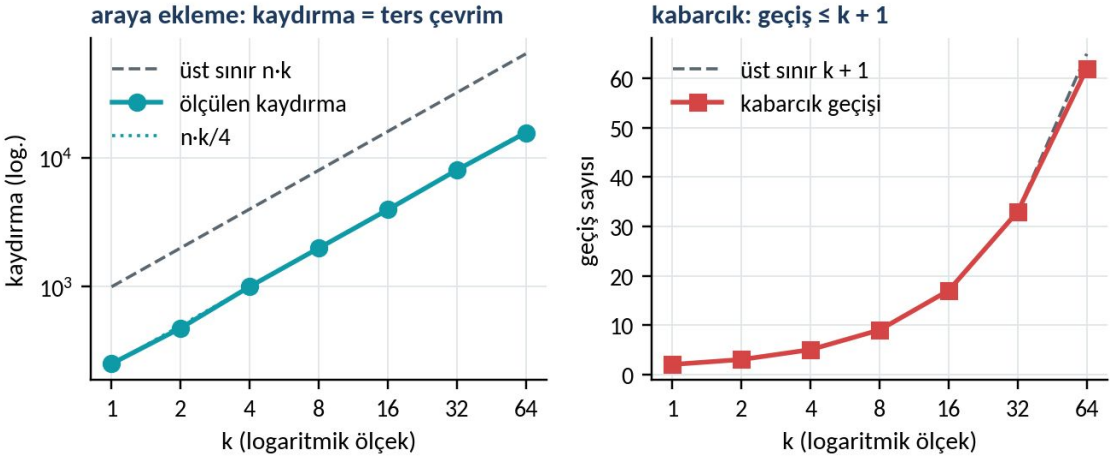
Her öğenin sıralı konumundan en çok k uzakta olduğu diziye **k-sıralı** (neredeyse sıralı) dizi denir: zaman damgasına göre gelen ama ağdaki gecikme yüzünden biraz karışmış olaylar, ya da sıralı bir listeye birkaç değişiklik yapıldıktan sonraki durum. Örnek 11, 1..1000 dizisini (k + 1)'lik bloklar içinde karıştırarak k-sıralı diziler üretir (Fisher–Yates, ortak LCG) ve programla ölçülen en büyük uzaklığı yazar.

Bir öğenin yaptığı kaydırma, solundaki kendisinden büyük öğe sayısıdır; k-sıralı dizide bunlar en çok k tanedir, yani ters çevrim sayısı ve kaydırma n·k ile sınırlıdır: araya ekleme O(nk) çalışır. Ölçülen kaydırma/(nk) oranı k = 1 … 64 için 0,239 ile 0,253 arasındadır (Şekil 5.15): blok içi rastgele karıştırmada beklenen değer n·k/4 civarındadır. Kabarcık sıralamasının geçiş sayısı en çok k + 1'dir (her geçişte bir öğe en çok bir adım sola gider), ama her geçiş yaklaşık n karşılaştırma yapar. Seçmeli sıralama ise her k'da tam 499 500 karşılaştırma yapar: uyarlanabilir değildir. Dizileri üreten fonksiyon (blok içi Fisher–Yates):

c/H05_11_k_sirali.c · satır 40–52

```c
static void k_sirali_dizi(int *a, size_t n, size_t k)
{
    for (size_t i = 0; i < n; i++) {
        a[i] = (int)(i + 1);
    }
    for (size_t bas = 0; bas < n; bas += k + 1) {
        size_t son = bas + k < n ? bas + k : n - 1;   /* blok: bas..son */
        for (size_t i = son; i > bas; i--) {          /* Fisher–Yates */
            size_t j = bas + (size_t)rastgele_aralik(0, (int)(i - bas));
            takas(&a[i], &a[j]);
        }
    }
}
```

Şekil 5.15
k-sıralı dizi, n = 1000 (Örnek 11): araya eklemenin kaydırması n·k'nın yaklaşık dörtte biri; kabarcığın geçiş sayısı ≤ k + 1; iki eksen de logaritmik



6. Haftanın Deneyi: Algoritma × Girdi Türü

Soru. Üç algoritma dört girdi türünde (rastgele, sıralı, ters sıralı, neredeyse sıralı) kaç karşılaştırma ve eleman hareketi yapar, ne kadar sürer? Kuramın öngörüleri (Tablo 5.5) ölçümle tutuyor mu? **Düzen.** Deney programları `kodlar/deney/{c,cpp,python}/H05_deney_siralama` üç dilde aynı algoritmaları sayaçlarla çalıştırır (`./H05_deney_siralama ALGORITMA GIRDI N [TEKRAR] [--csv]`), sürücü `olcum_calistir.py` bütün planı ölçer, `kestirim.py` oranları ve kestirimleri hesaplar. Girdiler: rastgele = 1..n'nin karıştırılmışı (ortak LCG, tohum 42), sıralı, ters sıralı, neredeyse sıralı (11'lik bloklar içinde karıştırılmış: her öğe en çok 10 konum uzakta). Ölçüm: 1 ısınma + 5 tekrar, ortanca (Sedgewick & Wayne, 2011); girdinin kopyalanması ölçüme girmez. Ortam: Intel(R) Xeon(R) Processor @ 2.80GHz; gcc 13.3.0; Python 3.11.15; -O2 (C ayrıca -O0, n = 5 000).

Paylaşılan makine. Ölçüm sırasında makinede başka işler de çalışıyordu (yük ortalaması: başta 1.13, sonda 4.77 (1 dk; 2 çekirdek)). Bu yüzden süre olarak sürecin **işlemci zamanı** (C/C++ `clock()`, Python `time.process_time`) esas alındı; duvar saati veri olarak tutuldu (≥ 1 ms ölçümlerde duvar/işlemci oranının ortancası 2,08). Bütün plan iki tur ölçüldü ve her yapılandırma için işlemci ortancası küçük olan tur seçildi. Karesel durumlar C'de n ≤ 20 000, C++'ta n ≤ 10 000, Python'da n ≤ 2 000'e kadar ölçüldü; Python'un n = 10 000 hücreleri **kestirimdir** (aşağıda "≈" ve "kest." ile işaretli).

6.1 Sayaçlar: ısı haritası

Tablo 5.7
n = 10 000: sayaçlar ve işlemci süresi ortancası (bu makinede; Python'un karesel hücreleri kestirim)

Algoritma	Girdi	Karşılaştırma	Takas / kaydırma	C -O2	C++ -O2	Python
seçmeli	rastgele	49 995 000	9 990	108,0 ms	109,4 ms	≈ 2,76 s (kest.)
seçmeli	sıralı	49 995 000	0	109,1 ms	111,5 ms	≈ 2,68 s (kest.)
seçmeli	ters	49 995 000	5 000	110,3 ms	112,0 ms	≈ 2,95 s (kest.)
seçmeli	neredeyse	49 995 000	7 201	107,4 ms	113,0 ms	≈ 3,00 s (kest.)
kabarcık	rastgele	49 974 294	24 716 796	149,2 ms	123,8 ms	≈ 5,57 s (kest.)
kabarcık	sıralı	9 999	0	9,0 µs	7,0 µs	0,633 ms
kabarcık	ters	49 995 000	49 995 000	64,0 ms	70,5 ms	≈ 7,87 s (kest.)

Algoritma	Girdi	Karşılaştırma	Takas / kaydırma	C -O2	C++ -O2	Python
kabarcık	neredeyse	109 934	24 875	0,296 ms	0,220 ms	9,9 ms
araya ekleme	rastgele	24 726 789	24 716 796	20,4 ms	17,7 ms	≈ 2,55 s (kest.)
araya ekleme	sıralı	9 999	0	12,0 µs	21,0 µs	0,998 ms
araya ekleme	ters	49 995 000	49 995 000	44,3 ms	52,8 ms	≈ 5,25 s (kest.)
araya ekleme	neredeyse	34 872	24 875	98,0 µs	0,113 ms	4,3 ms

Sayaçlar (Tablo 5.7 ve Şekil 5.16) kuramı **birebir** doğrular: seçmeli her girdide n(n − 1)/2 = 49 995 000 karşılaştırma yapar; kabarcık ve araya ekleme sıralı girdide n − 1 = 9 999 karşılaştırmayla biter; rastgele girdide kabarcığın takası ve araya eklemenin kaydırması aynı sayıdır (24 716 796 ters çevrim). Neredeyse sıralı girdide araya ekleme 34 872, kabarcık 109 934 karşılaştırma yapar; kabarcığınki ≈ 11 geçiş × n'dir (geçiş sayısı ≤ k + 1, k = 10).

Şekil 5.16
Algoritma × girdi türü, n = 10 000: karşılaştırma sayısı ve eleman hareketi (takas ya da kaydırma); renk ölçeği logaritmik, değer hücrede yazılı. Sayaçlar üç dilde aynıdır

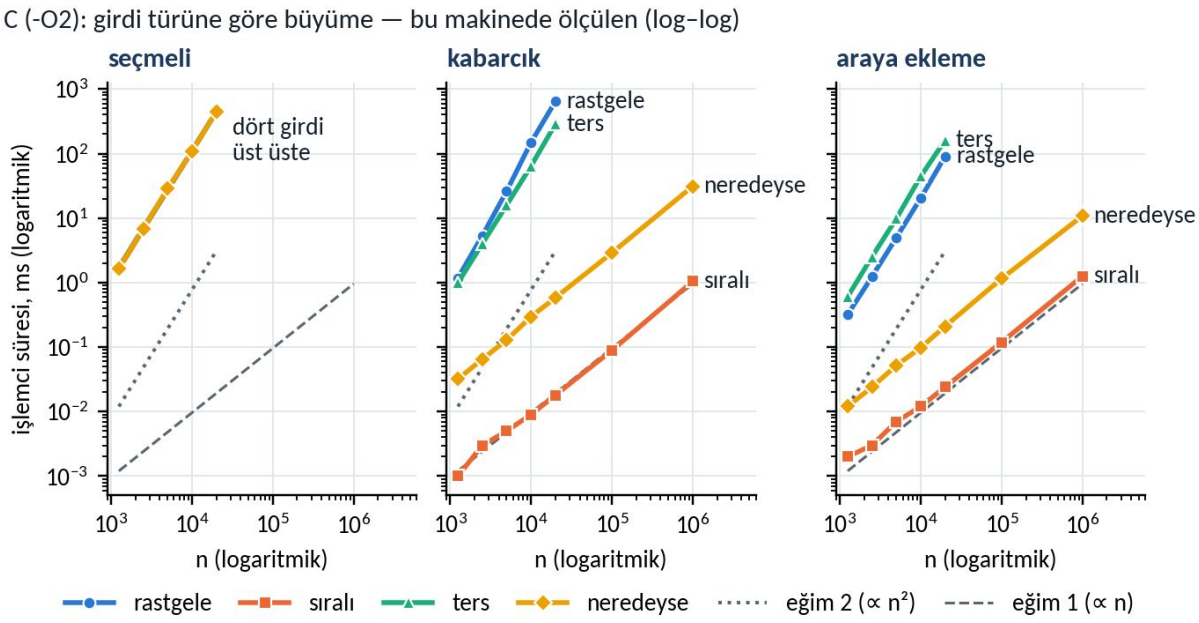


6.2 Süreler ve büyüme

Tablo 5.7 sürelerin de değerlerini verir; süre ısı haritası sunumda ve web sayfasındadır.

Aynı Θ(n²), farklı sabit. Rastgele girdide araya ekleme en hızlı, kabarcık en yavaştır (C'de kabarcık/araya = 7,33). Seçmeli dört girdide de yaklaşık aynı sürer. **Sayı fazla, süre az.** C'de kabarcık ters sıralı girdide rastgele girdinin 2,02 katı takas yaptığı hâlde 0,43 katı sürede biter: daha çok iş, daha kısa süre. Olası bir açıklama dal tahminidir: ters sıralıda her karşılaştırma takasla sonuçlanır ve işlemci bunu kolay öngörür, rastgele girdide takas kararı her adımda yazı tura gibidir. Bunu bu deneyde **ölçmedik**; bir yorum olarak not ediyoruz. **Uyarlanabilirlik.** Araya ekleme rastgele girdide sıralı girdiden 1 696 kat yavaştır.

Şekil 5.17
Süre ~ n, C -O2, log–log; dört girdi türü; eğim 1 ve 2 başvuru çizgileri



Oran testi. n iki katına çıkınca $\Theta(n^2)$ bir algoritmanın süresi yaklaşık 4 katına çıkmalıdır. Karesel durumlarda 80 ikiye katlamanın T(2n)/T(n) ortancası 4,13'tür (aralık 2,54–7,30); doğrusal durumlarda (≥ 0,05 ms) 16 katlamanın ortancası 2,09'dur (kuram 2). Şekil 5.17 içinde karesel eğriler eğim 2, sıralı ve neredeyse sıralı girdideki kabarcık ve araya ekleme eğim 1 çizgisine paraleldir.

Kestirim ve doğrulama. Ölçemediğimiz büyüklükler için kuramı kullandık: $T(n) = T(n_0)\cdot(n/n_0)^2$, n_0 = en büyük ölçülen n. Yöntemi sınamak için üç doğrulama ölçümü (tek tekrar) yaptık:

Tablo 5.8
Kestirim yönteminin sınanması (bu makinede)

Dil	Algoritma / girdi	n (n_0)	Kestirim (b = 2)	Ölçülen	Hata	Uydurulan eğim $\hat{b}$
Python	araya ekleme / rastgele	4 000 (2 000)	408,5 ms	468,5 ms	−%12,8	2,118
Python	seçmeli / rastgele	4 000 (2 000)	441,0 ms	488,9 ms	−%9,8	2,110
C	seçmeli / rastgele	40 000 (20 000)	1,78 s	1,78 s	%0,1	2,002

C'de b = 2 kestirimi neredeyse tam tutar. Python'da iki kat ileriye yapılan kestirim %10–13 eksik kalır: karşılaştırma başına maliyet n büyüdükçe hafifçe artmaktadır. Bu yüzden aşağıdaki kestirimler **alt kestirimdir** (işlem başı maliyet sabit varsayılır).

6.3 Büyük Sorunun sayısal yanıtı: bir milyon tamsayı

Şekil 5.1 (Bölüm 1) n = 10^6 rastgele tamsayı için bu makinedeki sayıları toplar. Kestirimler (n ≤ 20 000 ölçümlerinden T ∝ n^2; kabarcık, seçmeli, araya ekleme): C'de (-O2) ≈ 26,7 dk, ≈ 18,6 dk, ≈ 3,8 dk; C++'ta ≈ 20,6 dk, ≈ 18,2 dk, ≈ 2,9 dk; Python'da ≈ 15,5 sa, ≈ 7,7 sa, ≈ 7,1 sa. Kütüphane sıralamalarının ölçülen süreleri: `qsort` 153,9 ms, `std::sort` 75,1 ms, `list.sort` 752,8 ms.

C'de kabarcık sıralaması `qsort`'tan ≈ 10 409 kat, araya ekleme ≈ 1 466 kat yavaştır. "Kabarcık sıralaması yanlış yol olur" sözünün arkasındaki sayı budur; ama aynı sayılar daha iyisinin de (araya ekleme) yanlış yol olduğunu söyler: bir milyon öğede sorun algoritmanın sabiti değil, $\Theta(n^2)$ büyüme sınıfıdır. O(n log n) algoritmaları H6'da yazacağız.

6.4 Dil sabitleri ve derleme düzeyi

Karşılaştırma başına işlemci süresi Python'da C'nin 25–121 katıdır (rastgele girdi; Python n = 2 000, C n = 1 250: ortak bir n ölçülmedi). C++ ile C arasında belirgin bir fark yoktur (n = 10 000 oranları: seçmeli 1,01, kabarcık 0,83, araya ekleme 0,87; paylaşılan makinedeki gürültüyle karışıktır). Eniyileme kapatıldığında (-O0, n = 5 000) kabarcık 2,2, araya ekleme 3,8 kat yavaşlar; seçmelide fark küçüktür (1,05).

6.5 Sınırlılıklar ve açık kalan soru

- **Kabarcık sıralamasının oran sapması (açık).** C ve C++'ta kabarcık/rastgele ikiye katlamaları 4,30–5,72 verdi; kuram 4 der. Karşılaştırma başına süre n büyüdükçe artıyor (C -O2: n = 1 250 → 1,51 ns, n = 2 500 → 1,71 ns, n = 5 000 → 2,09 ns, n = 10 000 → 2,98 ns, n = 20 000 → 3,21 ns), ters sıralı girdide ise neredeyse sabit kalıyor. Dizi n = 10 000'de bu makinenin çekirdek başına 32 KiB L1 veri önbelleğini aşar, ama ters sıralıda da aşar; yalnızca önbellek açıklaması yetmez. Dal tahmini yanılgılarının bedelinin büyük n'de önbellek kaçırmalarıyla birlikte arttığı bir olasılıktır; bunu ölçmeden bilemeyiz. Bu soruyu açık bırakıyoruz.
- **Gürültü.** Makine paylaşılan, 2 çekirdekli bir sanal makineydi; işlemci zamanı da önbellek paylaşımından etkilenir. Python'daki bazı aykırı oranlar büyük olasılıkla tek bir gürültülü ölçümden gelir.
- **Kestirimler** işlem başı maliyeti sabit sayar (alt kestirim); doğrulama ölçümleri tek tekrarlıdır. Sayaçlar ölçülen kodun içindedir (C'de yazmaçta tutulur, Python'da ek yük getirir).
- **Kendi makinenizde tekrarlayın:** `cd kodlar/deney && python3 olcum_calistir.py && python3 kestirim.py` (bu makinede birkaç dakika). Tek bir hücre için: `./H05_deney_siralama araya rastgele 10000 5`. Sayaçlarınız aynı, süreleriniz farklı çıkmalıdır.

7. Hata Avcısı

Aşağıdaki on bir hatanın her biri `kodlar/hata_avcisi/` altında kasıtlı hatalı bir dosya ve düzeltilmiş eşiyle durur; gözlenen derleyici iletileri ve çıktılar `hata_avcisi/ciktilar/` altındadır. Tanımsız davranış içeren hatalarda gözlenen belirti **bu derleyicide ve bu çalıştırmada** gözlenendir; başka bir derleyici, eniyileme düzeyi ya da çalıştırma başka bir şey yapabilir. Tablonun ardından on bir hatanın kasıtlı hatalı satırları gelir (sarı, dosyanın satır numaralarıyla).

Tablo 5.9
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Seçmelide iç döngüde takas	sonuç doğru; ters sıralı n = 1000'de 499 500 takas (düzeltilmiş: 500)	algoritma artık seçmeli değil; “en çok n − 1 takas” kaybolur	iç döngüde yalnızca `en_kucuk = j`; takas dışarıda
2	Araya eklemede `a[j] > anahtar && j >= 0`	C: uyarı yok, çıktı doğru görünür; ASan: heap-buffer-overflow (a[−1]). Python: hata yok, `a[-1]` son öğeyi okur	kısa devre soldan sağa: sınır denetimi önce gelmeli (C'de tanımsız davranış)	`j >= 0 && a[j] > anahtar`
3	Karşılaştırıcıda `x - y` ve `(int)(x - y)`	uyarı yok; int dizisi “0 3 7 2147483647 -2147483648 -1” sırasında; UBSan: signed integer overflow	işaretli taşma tanımsız davranış; kesme farkı 1'den küçük çiftleri “eşit” sayar, eşitlik geçişsizleşir (ISO/IEC, 2018, 7.22.5)	`(x > y) - (x < y)`

#	Hata	Belirti (bu makinede)	Neden	Çözüm
4	qsort'a `sizeof(a)` (işaretçi)	uyarı yok; "sonuç: 0 0 8 31 25 4 39 12", stdout tamponu bile bozuldu; ASan: heap-buffer-overflow	8 baytlık öğe sanılır, dizinin dışına yazılır (tanımsız davranış)	`sizeof a[0]` ya da `sizeof *a`
5	Dizgi karşılaştırıcısının imzası `(const char *, const char *)`	GCC 13: `-Wincompatible-pointer-types` uyarısı (GCC 14 ve sonrası: hata; GNU Project, t.y.); sıralanmamış çıktı	qsort öğenin ADRESİNİ verir (`char **`); uyumsuz çağrı tanımsız davranış	`(const void *, const void *)` + `*(const char *const *)p`
6	qsort'un kararlı olduğunu varsaymak	glibc'de doğru görünür; standarda uygun kararsız bir qsort'ta yanlış	C standardı eşitlerin sırasını belirtmez (ISO/IEC, 2018, 7.22.5.2); "bu girdide çalıştı" kanıt değildir	tek, tam karşılaştırıcı
7	C++'ta `std::sort` ile ardışık geçişler	n = 16'da bozulma yok, n = 24'te 10 bozuk çift	`std::sort` kararsızdır	`std::stable_sort` ya da tek yüklem
8	`strcmp` ile Türkçe adlar	Çelik, Öztürk, Ünal, İnce, Şahin listenin sonunda	UTF-8'de Türkçe harflerin ilk baytı ASCII'den büyük	kod noktası çöz + Türkçe sıra tablosu (tam sürüm: Mini-OBS 2 `turkce.c`)
9	Python'da `x = x.sort()`	`None`; ardından TypeError	`list.sort()` yerinde sıralar, `None` döndürür	`x.sort()` ya da `y = sorted(x)`
10	Kabarcıkta `j < n - i`	C: diziye 0 girdi, 42 dışarı itildi; ASan: heap-buffer-overflow. Python: IndexError	`a[j + 1]` = `a[n]` okunur ve yazılır (C'de tanımsız davranış)	`j + 1 < n - i`
11	Erken çıkış bayrağı iç döngüde sıfırlanıyor	1 geçişte durur, dizi sıralı değil (HA1 ve HA11: belirti sayaçta)	bayrak "son karşılaştırmada takas oldu mu?" anlamına gelir	bayrağı her geçişin başında bir kez sıfırla

C `hata_avcisi/H05_HA1_secmeli_ic_takas.c · satır 25–29`

```c
    for (size_t i = 0; i + 1 < n; i++) {
        for (size_t j = i + 1; j < n; j++) {
            if (a[j] < a[i]) {
                takas(&a[i], &a[j]);    /* HATA: iç döngüde takas */
            }
```

C `hata_avcisi/H05_HA2_kosul_sirasi.c · satır 22–22`

```c
        while (a[j] > anahtar && j >= 0) {   /* HATA: koşul sırası */
```

C `hata_avcisi/H05_HA3_fark_karsilastirici.c · satır 21–31`

```c
{
    int x = *(const int *)p;
    int y = *(const int *)q;
    return x - y;                   /* HATA: taşabilir */
}

static int double_artan(const void *p, const void *q)
{
    double x = *(const double *)p;
    double y = *(const double *)q;
    return (int)(x - y);            /* HATA: kesme, 0.3 - 0.1 -> 0 */
```

C **hata_avcisi/H05_HA4_qsort_oge_boyutu.c · satır 33–34**

```c
    printf("sizeof(a) = %zu, sizeof(a[0]) = %zu\n", sizeof(a), sizeof(a[0]));
    qsort(a, n, sizeof(a), int_artan);       /* HATA: işaretçinin boyutu */
```

C **hata_avcisi/H05_HA5_karsilastirici_imzasi.c · satır 19–21**

```c
static int dizgi_artan(const char *x, const char *y)   /* HATA: imza */
{
    return strcmp(x, y);           /* HATA: x, y öğenin adresidir */
```

C **hata_avcisi/H05_HA6_qsort_kararlilik.c · satır 64–65**

```c
    sirala(a, n, sizeof a[0], ad_artan);         /* 1. geçiş: ad */
    sirala(a, n, sizeof a[0], ortalama_azalan);  /* 2. geçiş: ortalama */
```

C++ **hata_avcisi/H05_HA7_kararsiz_gecisler.cpp · satır 41–46**

```cpp
    std::sort(a.begin(), a.end(),                 // 1. geçiş: ad
              [](const Ogrenci& x, const Ogrenci& y) { return x.ad < y.ad; });
    std::sort(a.begin(), a.end(),                 // 2. geçiş: ortalama
              [](const Ogrenci& x, const Ogrenci& y) {
                  return x.ortalama > y.ortalama;
              });
```

C **hata_avcisi/H05_HA8_strcmp_turkce.c · satır 16–19**

```c
static int dizgi_artan(const void *p, const void *q)
{
    return strcmp(*(const char *const *)p, *(const char *const *)q);
}
```

Python **hata_avcisi/H05_HA9_sort_none.py · satır 12–14**

```python
    notlar = notlar.sort()               # HATA: sort() None döndürür
    print("sıralı notlar:", notlar)
    print("en düşük not:", notlar[0])
```

C **hata_avcisi/H05_HA10_dongu_siniri.c · satır 26–27**

```c
        for (size_t j = 0; j < n - i; j++) {     /* HATA: bir fazla */
            if (a[j] > a[j + 1]) {
```

C **hata_avcisi/H05_HA11_erken_cikis_bayragi.c · satır 24–31**

```c
    int takas_oldu = 0;
    for (size_t i = 0; i + 1 < n; i++) {
        for (size_t j = 0; j + 1 < n - i; j++) {
            takas_oldu = 0;                  /* HATA: yanlış yer */
            if (a[j] > a[j + 1]) {
                takas(&a[j], &a[j + 1]);
                takas_oldu = 1;
            }
```

C'de dizinin dışına erişim (HA2, HA4, HA10), işaretli taşma (HA3) ve uyumsuz imzalı fonksiyonla çağrı (HA5) **tanımsız davranıştır**: bu çalıştırmalarda HA2'nin çıktısı doğru göründü, HA4'te stdout tamponu bile bozuldu; hepsini ASan ya da UBSan yakaladı. Python'da `a[-1]` geçerlidir (son öğe), `a[n]` ise `IndexError` fırlatır. Kural: önce sınır, sonra öğe.

8. Dil Penceresi: C · C++ · Python

Algoritma üç dilde aynıdır; değişen gösterim, kütüphane güvenceleri ve bellek modelidir. Tablo 5.10 haftanın dil farklarını toplar; her örneğin ayrıntılı dil notu Ek A'dadır.

Tablo 5.10
Haftanın dil farkları

Öğe	C	C++	Python
Takas	geçici değişkenli `takas(int *, int *)`	`std::swap(a[i], a[j])`	`a[i], a[j] = a[j], a[i]`
Dizi ve boyut	`int *a, size_t n` (işaretsiz)	`std::vector<int>& + size()`	`list + len(a)`
−1'e inen sayaç	`ptrdiff_t j` (işaretli)	`auto j = static_cast<std::ptrdiff_t>(i) - 1`	`j = i - 1` (int işaretli)
Kütüphane sıralaması	`qsort`: algoritma ve kararlılık belirtilmez	`std::sort` kararsız, O(n log n) karşılaştırma; `std::stable_sort` kararlı	`list.sort()` / `sorted()`: kararlı (Timsort tabanlı)
Karşılaştırma biçimi	`int kars(const void *, const void *)`: < 0, 0, > 0	`bool` yüklem: "x, y'den önce mi?" (katı zayıf sıralama)	`key=` anahtar fonksiyonu, `reverse=True`; `functools.cmp_to_key`
Çok anahtar	anahtarları sırayla dene: `if (k != 0) return k;`	`std::tie(…) < std::tie(…)` ya da sırayla `!=`	demet anahtar: `key=lambda o: (-o.ortalama, o.soyad)`
Yerinde mi, yeni mi?	`qsort` yerinde	`std::sort` yerinde	`a.sort()` yerinde ve `None` döndürür; `sorted(a)` yeni liste
Tamsayı farkı	`x - y` taşabilir (tanımsız davranış)	aynı (C ile aynı kurallar)	taşmaz (sınırsız tamsayı)
Dizgi sırası	`strcmp`: bayt sırası	`std::string <`: bayt sırası	`str <`: kod noktası sırası (UTF-8'de bayt sırasıyla aynı)
Hizalı yazım	`printf("%-14s")` bayt sayar	`std::setw` bayt sayar	`f"{s:<14}"` karakter sayar

C++'ta Python'un demet anahtarının karşılığı `std::tie`'dır: başvurulardan kurulan demetler sözlük sırasıyla karşılaştırılır; azalan anahtarda iki taraf yer değiştirir: `std::tie(y.ortalama, x.soyad, x.ad) < std::tie(x.ortalama, y.soyad, y.ad)`. Kavram 2 (`kodlar/kavram/H05_K02_std_tie.cpp`) Örnek 10'un ölçütünü böyle yazar (kâhin: `std::stable_sort`); çıktısı (`std::tie sırası: 108 104 110 103 105 107 101 106 109 102`) Örnek 10'un A sırasıyla aynıdır.

Kütüphane karşılıkları (yalnızca kâhin ve not olarak): C `qsort`; C++ `std::sort`, `std::stable_sort`, `std::greater`, sıralı vektöre ekleme için `std::upper_bound` + `insert`; Python `sorted`,

`list.sort`, `key=`, `reverse=`, `functools.cmp_to_key`, sıralı listeye ekleme için `bisect.insort` (Python Software Foundation, t.y.). Her örnek dosyası kütüphane karşılığını son satırındaki yorumda verir.

9. Mühendis Gözüyle: Kütüphaneyi Kullan, Ama Güvencesini Oku

Gerçek bir programda sıralama algoritmasını kendimiz yazmayız; kütüphaneyi çağırırız. Bu haftanın mühendislik dersi, kütüphaneyi kullanmadan önce üç soruyu belgesinden yanıtlamaktır: **kararlılık** güvencesi var mı, **karmaşıklık** güvencesi var mı, **karşılaştırıcı sözleşmesi** nedir?

- **C:** `qsort` için standart ne algoritmayı ne karmaşıklığı ne de kararlılığı belirtir: "eşit karşılaştırılan iki öğenin sonuçtaki sırası belirtilmemiştir" (ISO/IEC, 2018, 7.22.5.2); karşılaştırıcı tutarlı bir tam sıralama tanımlamalıdır (7.22.5; Hata Avcısı 3). Bu makinedeki glibc'nin eşitleri koruması bir **gözlemdir**, güvence değildir (Hata Avcısı 6).
- **C++:** `std::sort` O(n log n) karşılaştırma güvencesi verir, kararlılık güvencesi vermez; `std::stable_sort` kararlıdır (ISO/IEC, 2017). Yüklem katı zayıf sıralama olmalıdır (`<`, `<=` değil).
- **Python:** `list.sort` ve `sorted` için kararlılık belgelenmiş bir güvencedir (Python Software Foundation, t.y.); kararlı ardışık geçişler güvenle kullanılabilir. `key=` her öğe için bir kez hesaplanır.
- **Derleyiciyi ve kâhini dinleyin:** yanlış imzalı karşılaştırıcı GCC 13'te uyarı, GCC 14'te hatadır (Hata Avcısı 5). Eşit anahtarlar kâhin karşılaştırmasını belirsizleştirir; Mini-OBS 2'nin kâhini ölçüte son anahtar olarak öğrenci numarasını ekler: tam sıralamada doğru sonuç tektir.

Kısacası: bir güvenceye dayanıyorsanız onu belgede bulun ve teste yazın; belgede yoksa ona güvenmeyin.

10. Mini-OBS 2 Adımı: v2.4 — Çok Ölçütlü Sıralı Listeler

Bu hafta. Mini-OBS'ye çok ölçütlü sıralı listeler ekliyoruz: başarı sırası (YANO ↓, eşitlikte soyad ↑, ad ↑), Türkçe alfabe sırasıyla soyad listesi ve ders ortalaması listeleri. Seçmeli, kabarcık ve araya ekleme sıralamasını qsort arayüzünde yazıyor, karşılaştırıcıları ve kararlılığı kâhinle sınıyor, üç algoritmayı dört girdi türünde sayıp ölçüyoruz.

10.1 Gereksinimler

- `turkce.h/.c`: `utf8_coz`, `tr_sira_degeri`, `tr_karsilastir` (birincil sıra + bayt ikincil ölçüt: tam sıralama), `tr_onek_mi`. Sıra: boşluk < ' < - < A B C Ç D E F G Ğ H I İ J K L M N O Ö P (Q) R S Ş T U Ü V (W X) Y Z; i → İ, ı → I.
- `siralama.h/.c`: `secmeli_sirala_say`, `kabarcik_sirala_say` (erken çıkışlı), `araya_ekleme_sirala_say`; arayüz `(void *taban, size_t n, size_t boyut, int (*kars)(const void *, const void *), Sayac *sy)`; karşılaştırma ve taşıma sayaçları; her fonksiyonda değişmez yorumu.
- `liste.h/.c`: en az `basari`, `soyad`, `no` ve bir ders ölçütü; her ölçütün numarasız ve numarayla tamamlanmış karşılaştırıcısı; liste `const Ogrenci *` dizisidir.
- Karşılaştırıcılar taşmaya güvenlidir: `(x > y) − (x < y)`; YANO tamsayı (× 100) karşılaştırılır.
- `listele`: `--olcut`, `--yontem` (araya, secmeli, kabarcik, qsort, cok_gecisli, hepsi), `--ilk`, `--denetle`; çıkış kodları 0/1/2.
- Menü 3 ve `ara`'nın soyad önek araması Türkçe sırayla.

- Testler: elle izlenmiş sayaçlar; kâhin olarak `qsort` + tam ölçüt; kararlılık testi (eşit kayıtlar numara sırasında); çok geçişli = tek karşılaştırıcı.
- Ölçüm raporu: üç algoritma × dört girdi türü (karışık, sıralı, ters, neredeyse sıralı) × n = 500 … ≥ 4 000 için karşılaştırma, taşıma ve süre tablosu, ısı haritası ve log–log grafik; `qsort` ile karşılaştırma; Türkçe ↔ bayt karşılaştırıcısının maliyeti.

10.2 İpuçları

- Karşılaştırıcıyı önce kâğıtta yazın: “a, b'den önce gelmeliyse negatif”. Azalan sıra için işaretleri değiştirin: `(x < y) − (x > y)`.
- qsort karşılaştırıcısına gelen `void *` bir **öğeyi** gösterir: işaretçi dizisinde öğe `const Ogrenci *`'dır, yani `*(const Ogrenci *const *)p`.
- Araya eklemede döngü koşulunun sırası önemlidir: `j > 0 && kars(a[j−1], anahtar) > 0` — önce sınır.
- Kararlılık için araya eklemede `> 0` ile kaydırın (eşitte durun); kabarcıkta yalnızca `> 0` ile takas edin.
- Türkçe harfleri bayt bayt değil karakter karakter (UTF-8 çözerek) karşılaştırın; `strcmp` yalnızca son eşitlik ölçütü olarak kalsın.
- Kâhin karşılaştırıcısına numarayı son anahtar olarak ekleyin: `qsort` kararlı değildir, ama tam bir sıralamada sonucu tektir.
- Ölçümde her çalıştırma başlangıç listesini kopyalasın; aksi hâlde ikinci çalıştırma zaten sıralı listeyi sıralar.

> MİNİ-OBS 2 PROJESİ • **Sayaç sözleşmesi farkı**
>
> Mini-OBS 2 v2.4 eleman hareketini **taşıma** (öğe kopyası) olarak sayar: bir takas 3 taşıma (geçici değişkenle), araya eklemede her kaydırma ve anahtarın alınıp konması birer taşımadır. Bu ders notunda ise takas ve kaydırmayı ayrı sayıyoruz. İki sözleşmenin sayıları birbirine dönüştürülebilir, ama doğrudan karşılaştırılamaz. Ayrıca Mini-OBS ölçümleri öğrenci kayıtlarıyla ve Türkçe karşılaştırıcıyla yapılır; Haftanın Deneyi tamsayılarla.

10.3 Kabul ölçütleri

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri ve `test_kahin` (300 deneme) geçer.
2. `./listele ornek_veri/ogrenciler.csv --olcut soyad --ilk 8` sırası: ÇELİK, ÇINAR, DOĞAN, KAYA, ÖZTÜRK, ŞAHİN, YILMAZ (AYŞE NUR), YILMAZ (CAN); `--olcut soyad_bayt` aynı veride DOĞAN ile başlar ve ÇELİK'i YILMAZ'lardan sonraya koyar.
3. `./listele ornek_veri/ogrenciler.csv --denetle`: sıra 2301010050 (EMRE ÇINAR, 3,18), 2301010045 (AYŞE NUR YILMAZ, 2,91), 2301010048 (CAN YILMAZ, 2,79), …, son satır 2301010046 (MEHMET ALİ KAYA, 1,26); “Kâhin (qsort + numara): AYNI”, “kararlılık: EVET”, çıkış 0.
4. `./listele testler/kararlilik.csv --yontem hepsi`: araya, kabarcik, qsort ve cok_gecisli için “kâhinle aynı EVET / EVET”; secmeli için “HAYIR / HAYIR”. `--yontem secmeli --denetle` çıkış kodu 1.
5. Sayaçlar (`olcer`, n = 1 000): `liste_araya --sira artan` → karşılaştırma 999, taşıma 1 998; `liste_secmeli` her `--sira` için karşılaştırma 499 500; `liste_kabarcik --sira artan` → karşılaştırma 999, taşıma 0.

6. `tr_karsilastir` en az şu çiftleri doğru sıralar: ÇELİK < DEMİR, İREM < JALE, ÖZTÜRK < PEKER, ŞEN < ÜNAL, IŞIK < İNCE, AYŞE NUR < AYŞEGÜL.
7. Menü 3'te `*` girdisi soyadları Türkçe sırayla listeler (ÇELİK ilk, YILMAZ son).
8. Kendi makinenizde n = 4 000 karışık başarı listesinde araya ekleme seçmeliden ve kabarcıktan hızlıdır; sıralı girdide araya ekleme karışık girdiden en az 50 kat hızlıdır (bu makinede 162 kat).
9. `--hata-goster kesme` ve `--hata-goster kararsiz` uyuşmazlığı basar ve 1 ile çıkar; `make asan` temizdir.

10.4 Test senaryoları

Tablo 5.11
Mini-OBS 2 v2.4 test senaryoları

#	Girdi	Beklenen çıktı / davranış
1	`listele ornek_veri/ogrenciler.csv --olcut soyad`	Ç… Ç… D… K… Ö… Ş… Y… Y… (Türkçe)
2	aynı, `--olcut soyad_bayt`	D… K… Y… Y… Ç… Ç… Ö… Ş… (bayt)
3	aynı, `--olcut basari --denetle`	3,18 (ÇINAR) · 2,91 (AYŞE NUR YILMAZ) · 2,79 (CAN YILMAZ) · … · 1,26 (KAYA); AYNI, EVET
4	`testler/kararlilik.csv --yontem araya`	ECE KAYA …061, …062, …066 (numara sırası)
5	`testler/kararlilik.csv --yontem secmeli --denetle`	ECE KAYA …066, …062, …061; FARKLI, HAYIR; çıkış 1
6	aynı, `--yontem cok_gecisli --denetle`	AYNI, EVET (kararlı geçişler)
7	`listele ... --olcut alg --ilk 4`	ALG ortalaması azalan: 96,80, 90,40, 88,35, 79,00
8	`listele ... --olcut yas`	kullanım iletisi, çıkış 2
9	`olcer liste_araya --sira artan --n 1000`	karşılaştırma 999, taşıma 1 998
10	`olcer liste_secmeli --sira artan --n 1000`	karşılaştırma 499 500
11	`test_kahin --hata-goster kesme`	KÂHİN UYUŞMAZLIĞI (ölçüt alg); çıkış 1
12	`test_kahin --hata-goster kararsiz`	KÂHİN UYUŞMAZLIĞI (ölçüt basari, secmeli); çıkış 1

Başlangıç noktası ve meydan okumalar Mini-OBS 2 v2.4 belgesindedir.

11. Alıştırmalar ve Atölyeler

Alıştırmalar yedi türdedir: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez yazma, D) hata bulma, E) Parsons, F) kod yazma, G) diller arası okuma. Kodlu alıştırmaların dosyaları `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** tahmin edin, sonra çalıştırın. Elle sayımları `kodlar/kavram/H05_K03_sayac_araci.c` ile sınayabilirsiniz (ör. `./H05_K03_sayac_araci 5 2 4 6 1 3`). Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 5.1 ★☆☆ *İzleme*

a = {5, 2, 4, 6, 1, 3} dizisini seçmeli, kabarcık (erken çıkışlı) ve araya ekleme sıralamasıyla elle izleyin: her geçişten (araya eklemede her adımdan) sonraki diziyi yazın; karşılaştırma, takas ve kaydırma sayılarını bulun. Dizinin ters çevrim sayısı kaçtır ve hangi sayılara eşittir?

Alıştırma 5.2 ★★☆ *Çıktı tahmini*

Program ne yazdırır (`yaz` diziyi tek satırda, her öğenin önünde bir boşlukla yazar)? `f` hangi algoritmadır? `f` takası koşulsuz yapar: en büyük zaten yerindeyse öğeyi kendisiyle değiştirir. Bu girdide kaç geçişte gerçek bir yer değiştirme olur?

alistirmalar/H05_A01_cikti_tahmini.c · satır 19–40

```c
static void f(int *a, size_t n)
{
    for (size_t son = n; son > 1; son--) {
        size_t k = 0;
        for (size_t j = 1; j < son; j++) {
            if (a[j] > a[k]) {
                k = j;
            }
        }
        int g = a[k];
        a[k] = a[son - 1];
        a[son - 1] = g;
        yaz(a, n);
    }
}

int main(void)
{
    int a[] = {5, 1, 4, 2, 3};
    f(a, sizeof a / sizeof a[0]);
    return 0;
}
```

11.2 Karmaşıklık (B)

Alıştırma 5.3 ★☆☆ *Karmaşıklık*

Seçmeli sıralamanın karşılaştırma sayısının her girdide $n(n-1)/2$ olduğunu toplam yazarak gösterin. $n = 10^5$ için kaç karşılaştırma yapılır? Bu makinede C'de (-O2) karşılaştırma başına ≈ 2,16 ns ölçüldü ($n = 10\,000$); $n = 10^5$ için süreyi kestirin ve kestirimin hangi varsayıma dayandığını yazın.

Alıştırma 5.4 ★★☆ *Karmaşıklık*

n çift olmak üzere girdi {2, 1, 4, 3, 6, 5, …, n, n − 1} olsun (komşu çiftler yer değiştirmiş). Ters çevrim sayısı kaçtır? Araya eklemenin kaydırma ve karşılaştırma sayılarını, kabarcık sıralamasının geçiş ve karşılaştırma sayısını ve seçmeli sıralamanın takas sayısını n cinsinden bulun. Hangisi $\Theta(n)$, hangisi $\Theta(n^2)$?

Alıştırma 5.5 ★★☆ *Karmaşıklık*

Rastgele bir permütasyonda (her sıralama eşit olasılıklı) beklenen ters çevrim sayısının $n(n-1)/4$ olduğunu gösterin (ipucu: her (i, j) çifti için ters olma olasılığı). Örnek 5'in üçüncü bölümündeki oran sütununu bu sonuçla yorumlayın: neden n = 10'da oran 1'den çok sapıyor da n = 1000'de sapmıyor?

11.3 Değişmez yazma (C)

Alıştırma 5.6 ★★☆ *Değişmez yaz*

Erken çıkışlı kabarcık sıralamasının dış döngüsü için bir değişmez yazın ve başlatma, koruma, sonlanma adımlarıyla gösterin. Ayrıca şunu kanıtlayın: bir geçişte hiç takas olmazsa dizi sıralıdır. Bu ikinci iddia değişmezden mi gelir, yoksa ayrı bir gözlem mi?

Alıştırma 5.7 ★★☆ *Değişmez yaz*

Araya eklemenin **iç** döngüsü (`while (j >= 0 && a[j] > anahtar)`) için bir değişmez yazın: döngünün her denetiminde boşluk nerededir, a[j + 2..i] hakkında ne söylenebilir, a[0..j] hakkında ne söylenebilir? Bu değişmezden dış döngünün değişmezinin korunduğunu çıkarın.

11.4 Hata bulma (D)

Alıştırma 5.8 ★★☆ *Hata bul*

Program anahtara göre doğru sıralıyor, ama araya ekleme sıralamasının Bölüm 3.4'teki özelliklerinden biri kayboldu. Hangisi? Çalıştırıp çıktıdaki harflere bakın. Hatalı satırı bulun ve tek karakter değiştirerek düzeltin.

alistirmalar/H05_A02_hata_bul_araya.c · satır 17–28

```
17  static void araya_ekleme_sirala(Kayit *a, size_t n)
18  {
19      for (size_t i = 1; i < n; i++) {
20          Kayit k = a[i];
21          ptrdiff_t j = (ptrdiff_t)i - 1;
22          while (j >= 0 && a[j].anahtar >= k.anahtar) {
23              a[j + 1] = a[j];
24              j--;
25          }
26          a[j + 1] = k;
27      }
28  }
```

Alıştırma 5.9 ★★☆ *Hata bul*

Bu seçmeli sıralama {3, 9, 1, 7, 4} dizisinde yanlış sonuç veriyor. Çıktıdaki ilk yanlış satır hangisi? Hatalı satırı düzeltin ve değişmezin (Bölüm 4.1) ilk hangi geçişte bozulduğunu açıklayın.

alistirmalar/H05_A03_hata_bul_secmeli.c · satır 20–35

```
20  static void secmeli_sirala(int *a, size_t n)
21  {
22      for (size_t i = 0; i + 1 < n; i++) {
23          size_t en_kucuk = 0;
24          for (size_t j = i + 1; j < n; j++) {
25              if (a[j] < a[en_kucuk]) {
26                  en_kucuk = j;
27              }
28          }
29          int g = a[i];
30          a[i] = a[en_kucuk];
31          a[en_kucuk] = g;
```

```
32          printf("i = %zu:", i);
33          yaz(a, n);
34      }
35  }
```

11.5 Parsons (E)

Alıştırma 5.10 ★★☆ *Parsons*

Aşağıdaki satırlar erken çıkışlı kabarcık sıralamasının karıştırılmış satırlarıdır (dosya tamamlanacak; bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın; sonucu Örnek 2'nin `kabarcik_sirala` fonksiyonuyla (sayaçlar hariç) karşılaştırın.

alistirmalar/H05_A04_parsons_kabarcik.c · satır 6–20

```
            takas(&a[j], &a[j + 1]);
    for (size_t i = 0; i + 1 < n; i++) {
        if (!takas_oldu) {
            takas_oldu = 1;
static void kabarcik_sirala(int *a, size_t n)
        int takas_oldu = 0;
            break;
        for (size_t j = 0; j + 1 < n - i; j++) {
            if (a[j] > a[j + 1]) {
{
}
}
}
}
}
```

11.6 Kod yazma (F)

Alıştırma 5.11 ★☆☆ *Kod yaz*

Diziyi **azalan** sıraya dizen bir seçmeli sıralama yazın (her geçişte kalanların en büyüğünü seçin) ve takas sayısını sayın. Sıralı, ters sıralı ve ortak dizi {42, 17, 8, 31, 25, 4, 39, 12} için takas sayılarını yazdırın. C++ ve Python karşılıklarını da yazın; üç çıktı aynı olmalıdır.

Alıştırma 5.12 ★★☆ *Kod yaz*

Çift yönlü kabarcık (kokteyl) sıralamasını yazın: bir geçiş soldan sağa (en büyüğü sona), bir geçiş sağdan sola (en küçüğü başa) gider; iki yönde de takas olmazsa durun. Örnek 2'nin beş girdisinde (sıralı, neredeyse sıralı, tavşan, kaplumbağa, ters sıralı) geçiş ve karşılaştırma sayılarını Örnek 2'nin tablosuyla karşılaştırın. Kaplumbağa sorunu çözüldü mü?

Alıştırma 5.13 ★★☆ *Kod yaz*

İkili aramalı araya ekleme yazın: anahtarın yerini a[0..i − 1] içinde H4'teki **üst sınır** (upper_bound: anahtardan büyük ilk öğe) aramasıyla bulun, sonra büyükleri kaydırın. n = 1000 rastgele permütasyonda (Örnek 4'ün girdisi) karşılaştırma ve kaydırma sayılarını Örnek 4 ile karşılaştırın. Neden alt sınır (lower_bound) değil üst sınır kullanmalıyız? (İpucu: Bölüm 3.4.)

Alıştırma 5.14 ★★★ *Kod yaz*

Genel bir **kararlılık sınayıcısı** yazın: `int kararli_mi(void (*sirala)(Kayit *, size_t), int deneme)` rastgele (anahtar 0..3, sıra no) kayıtları üretir, verilen fonksiyonla anahtara göre sıralar ve eşit anahtarlıların sıra numaralarının artan kalıp kalmadığını denetler; ilk bozulmada karşı örneği yazdırır. Seçmeli, kabarcık, araya ekleme ve Shell sıralamasıyla çalıştırın. Kararsız bir algoritma için kaç denemede karşı örnek bulunuyor?

11.7 Diller arası okuma (G)

Alıştırma 5.15 ★★☆ *Diller arası okuma*

Python programının üç satırlık çıktısını tahmin edin. `reverse=True` ile `key=lambda x: -x[1]` aynı sonucu verir mi? Eşit notlu Ali ile Ece'nin ve Can ile Efe'nin sırası ne olur? C'de aynı azalan sırayı `qsort` ile kurarsanız aynı güvenceyi alır mısınız?

alistirmalar/H05_A05_reverse_kararlilik.py · satır 7–18

```
notlar = [("Ali", 80), ("Can", 90), ("Ece", 80), ("Efe", 90),
          ("Naz", 70)]


def main():
    print(sorted(notlar, key=lambda x: x[1]))
    print(sorted(notlar, key=lambda x: x[1], reverse=True))
    print(sorted(notlar, key=lambda x: -x[1]))


if __name__ == "__main__":
    main()
```

Alıştırma 5.16 ★★☆ *Diller arası okuma*

Aşağıdaki C++ ve Python parçaları “sıralı bir kopya” almaya çalışıyor (`yaz` C++'ta vektörü adıyla birlikte tek satırda yazar). İkisi `a` ve `b`'yi yazdırdığında ne çıkar? Fark nereden geliyor (Tshukudu & Cutts, 2020)? Python'da özgün listeyi bozmadan sıralı kopya almanın iki yolunu yazın.

alistirmalar/H05_A06_kopya_siralama.cpp · satır 23–27

```
    std::vector<int> a = {42, 17, 8, 31};
    std::vector<int> b = a;
    std::sort(b.begin(), b.end());
    yaz("a", a);
    yaz("b", b);
```

Python **alistirmalar/H05_A06_kopya_siralama.py · satır 9–13**

```
    a = [42, 17, 8, 31]
    b = a
    b.sort()
    print("a:", *a)
    print("b:", *b)
```

11.8 Atölye 1 · İnsan sıralama makinesi (gruplar, ≈ 20 dk)

ALGORİTMA ATÖLYESİ · **Kartlarla üç algoritma; karşılaştırmaları ve takasları sayın**

- Üç grup kurun; her gruba üzerinde 42, 17, 8, 31, 25, 4, 39, 12 yazan sekiz kart verin. Kartlar masada sırayla, yüzü kapalı durur. Gruplarda üç rol vardır: **makine** (yalnızca iki kartı açıp karşılaştırabilir ve iki kartın yerini değiştirebilir), **sayaç** (her karşılaştırmayı ve her takası ya da kaydırmayı sayar), **denetçi** (algoritmanın kuralına uyulduğunu denetler).
- 1. grup seçmeli, 2. grup kabarcık (erken çıkışlı), 3. grup araya ekleme sıralaması yapar. Başlamadan önce her grup kendi sayaçlarını **tahmin eder**.
- Bittiğinde sayılar tahtadaki tabloya (satırlar: grup; sütunlar: tahmin ve sayılan karşılaştırma, tahmin ve sayılan takas/kaydırma, program) yazılır ve `./H05_K03_sayac_araci` çıktısıyla (aynı sayılar, varsayılan girdi) karşılaştırılır. Fark varsa nerede saptınız? (Sık sapma: seçmelide en küçük zaten yerindeyken takas yapmak; kabarcıkta erken çıkış geçişini saymamak.)
- İkinci tur: aynı kartları sıralı ve ters sıralı dizip tekrarlayın. Hangi grubun sayısı değişmedi? Hangisininki 7'ye indi? Uyarlanabilirlik ve "en çok n − 1 takas" kavramlarını kendi sayılarınızla açıklayın.
- Tartışma: Karşılaştırma mı daha "yorucuydu", kart taşımak mı? Kartlar ağır kutular olsaydı hangi algoritmayı seçerdiniz (Bölüm 3.8)?

11.9 Atölye 2 · Sıralama laboratuvarı (eşli, ≈ 25 dk)

Her eş bir dili (C, C++ ya da Python) seçer ve `kodlar/deney/` programıyla üç algoritmayı üç girdi türünde ölçer. C ve C++ için n = 4 000 ve 8 000, Python için n = 1 000 ve 2 000 kullanın (karesel durumlar Python'da dakikalara uzamasın). Ölçüm komutu (C örneği; C++ ve Python'da aynı bağımsız değişkenler):

KOMUTLAR

```
$ cd kodlar/deney/c
$ gcc -std=c17 -Wall -Wextra -pedantic -O2 H05_deney_siralama.c ortak_rastgele.c
zamanlayici.c -o H05_deney_siralama
$ ./H05_deney_siralama araya rastgele 4000 5
$ python3 ../python/H05_deney_siralama.py araya rastgele 1000 5
```

- Tabloyu doldurun: satırlar algoritma × girdi (rastgele, sıralı, ters sıralı), sütunlar karşılaştırma, takas/kaydırma, süre (iki n için) ve oran T(2n)/T(n). Sayaçlarınızı Tablo 5.7 ile karşılaştırın: aynı mı? (Sayaçlar n'ye bağlıdır; n = 10 000 değerlerini kapalı biçimlerle kendi n'nize çevirin.)
- Oranlar kurama uyuyor mu (karesel durumlarda ≈ 4, doğrusal durumlarda ≈ 2)? Uymayanları ölçümü tekrarlayarak sınayın. Bu makinede ters sıralı girdide kabarcık rastgele girdiden hızlı ölçüldü (Bölüm 6.2); sizin makinenizde de öyle mi?
- Tahtada üç dilin sonuçlarını bir ısı haritasında birleştirin (renk: süre, logaritmik). Dil değişince en hızlıdan en yavaşa algoritma sırası değişiyor mu?

11.10 Atölye 3 · Karşılaştırıcı atölyesi (eşli, ≈ 25 dk)

Örnek 8–10'un on kaydı beş ölçüte göre sıralanacak. `H05_A07_olcut_atolyesi.c` iskeleti kayıtları, kararlı araya ekleme sıralamasını ve beş ölçütün boş karşılaştırıcılarını verir (dosya derlenir, ama karşılaştırıcılar şimdilik 0 döndürdüğü için hiçbir şey sıralanmaz). Her karşılaştırıcı qsort sözleşmesine uyar (Bölüm 3.5): x önce gelecekse negatif, sonra gelecekse pozitif, eşitse 0.

- Beş karşılaştırıcıyı yazın: (1) başarı (ortalama ↓, soyad ↑, ad ↑), (2) soyad ↑, ad ↑, (3) ad ↑, (4) numara ↓, (5) ortalama ↑ ama eşitlikte **karşılaştırıcıda numara olmadan** kayıt sırası. Her biri için çıktıdaki numara sırasını önce elle tahmin edin.
- (1)'in sonucunu Örnek 10'un A sırasıyla karşılaştırın. (5)'te kayıt sırasının korunması neye dayanıyor? Proje bağlantısı: kendi Mini-OBS 2 v2.4'ünüzle `listele ornek_veri/ogrenciler.csv --olcut basari|soyad|ad|no|alg --denetle` komutlarını çalıştırın; `--olcut soyad` ile `soyad_bayt` sıralarını karşılaştırın (Bölüm 10, kabul ölçütü 2). Tartışma: bir ölçütün tam sıralama olması için ne gerekir (Bölüm 9)?

12. Kendini Değerlendirme

- ☐ Seçmeli, kabarcık ve araya ekleme sıralamasını küçük bir dizide elle izleyebiliyor, döngü değişmezlerini başlatma, koruma ve sonlanma adımlarıyla gerekçelendirebiliyorum (4.1–4.3).
- ☐ Üç algoritmanın en iyi, ortalama ve en kötü durum karşılaştırma ve takas/kaydırma sayılarını yazabiliyorum (4.5).
- ☐ Bir dizinin ters çevrim sayısını bulup kabarcık takası ve araya ekleme kaydırmasıyla ilişkilendirebiliyorum (3.3).
- ☐ Kararlı, yerinde, uyarlanabilir ve çevrimiçi kavramlarını örnekle açıklayabiliyor, dört algoritmayı bu özelliklere göre sınıflandırabiliyorum (3.4).
- ☐ `qsort` için taşmaya güvenli bir karşılaştırıcı; C++'ta yüklem, Python'da `key=` yazabiliyorum (3.5).
- ☐ Çok anahtarlı bir ölçütü tek karşılaştırıcıyla ve kararlı ardışık geçişlerle kurabiliyor, kararsız bir algoritmanın neyi bozduğunu gösterebiliyorum (3.4, 3.6).
- ☐ Kendi sıralamamı kâhinle ve “sıralı mı?” denetimiyle sınayabiliyor, ikisinin farkını açıklayabiliyorum (5.1).
- ☐ Bir ölçüm tablosunu oran testiyle yorumlayabiliyor, kestirimin hangi varsayıma dayandığını söyleyebiliyorum (6).
- ☐ Hata Avcısı'ndaki on bir hatanın belirtisini ve nedenini anlatabiliyorum (7).

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Bu haftanın konuları nerede derinleşiyor?**

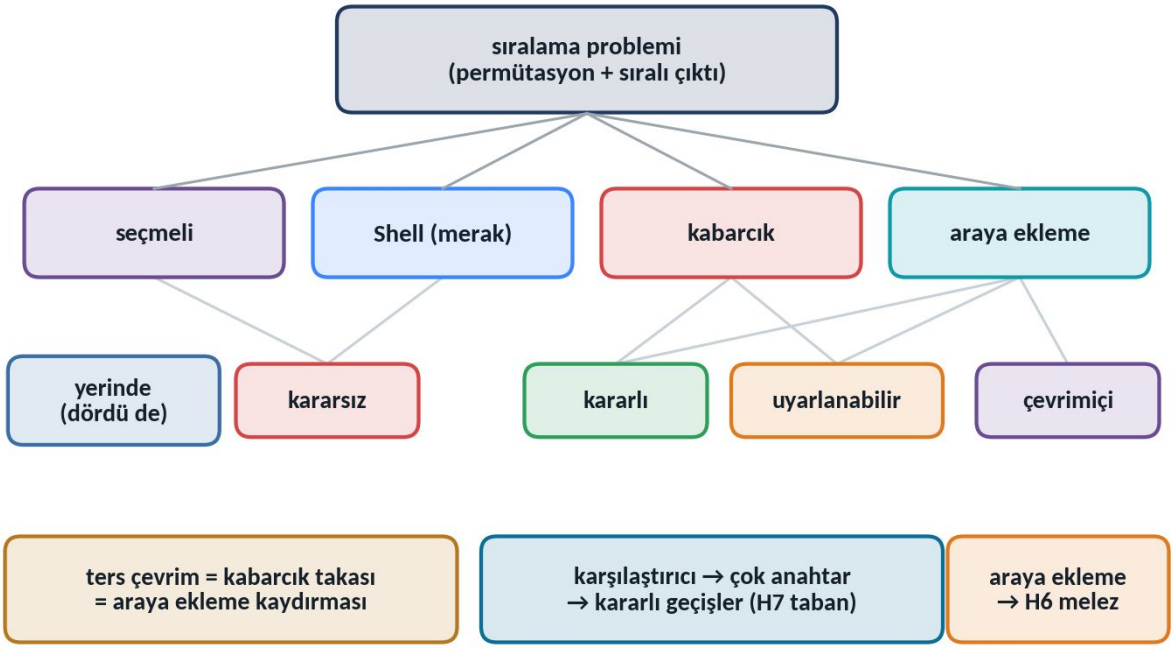
- **H6 · melez sıralama:** birleştirmeli ve hızlı sıralama Θ(n log n)'dir; küçük alt dizileri araya eklemeye devretmek onları hızlandırır (introsort, Timsort).
- **H6 · ters çevrim sayımı:** Örnek 5'te ters çevrimleri $\Theta(n^2)$'de saydık; birleştirmeli sıralamanın birleştirme adımına bir sayaç ekleyerek O(n log n)'de sayacağız. Bu haftanın sayımı orada kâhin olacak.
- **H7 · taban sıralaması:** en önemsiz basamaktan başlayan taban sıralaması (radix sort) ancak her basamak geçişi kararlıysa doğrudur: Örnek 10'un ardışık geçişlerinin karşılaştırmasız sürümü.

MERAK KUTUSU · **Karşılaştırmayla sıralamanın sınırı**

Yalnızca karşılaştırarak sıralayan **her** algoritma en kötü durumda Ω(n log n) karşılaştırma yapmak zorundadır: n öğenin n! sıralamasından doğru olanı seçmek gerekir ve her karşılaştırma olasılıkları en çok ikiye böler, yani en az $\log_2(n!) \approx n \log_2 n$ karşılaştırma gerekir (Cormen vd., 2022). H6'nın algoritmaları bu sınıra ulaşır; H7'nin sayma ve taban sıralaması karşılaştırma yapmadığı için sınırın dışındadır.

14. Özet, Kavram Haritası ve Terim Sözlüğü

Şekil 5.18
Haftanın kavram haritası: dört algoritma, dört özellik ve bağlantılar



- **Sıralama** çıktının hem sıralı hem de girdinin permütasyonu olmasını ister; test ikisini de sınamalıdır (kâhin).
- **Seçmeli:** her girdide n(n − 1)/2 karşılaştırma, en çok n − 1 takas; kararsız, uyarlanabilir değil.
- **Kabarcık (erken çıkışlı):** takas = ters çevrim; sıralıda n − 1 karşılaştırma; kararlı; zayıf uyarlanabilir (kaplumbağalar); pratikte en yavaşı.
- **Araya ekleme:** kaydırma = ters çevrim, süre Θ(n + ters çevrim); kararlı, uyarlanabilir, çevrimiçi; küçük ve neredeyse sıralı veride en iyisi.
- **Karşılaştırıcılar:** `(x > y) - (x < y)` taşmaz; çok anahtar için sözlük sırası ya da kararlı ardışık geçişler; Türkçe için bayt sırası yetmez.
- **Kütüphaneler:** `qsort` ve `std::sort` kararlılık güvencesi vermez; `std::stable_sort` ve Python'un sıralaması kararlıdır.
- **Bir milyon tamsayı:** Θ(n^2) algoritmalar bu makinede C'de 3,8 dk–26,7 dk, Python'da 7,1 sa–15,5 sa sürer (kestirim); kütüphane sıralamaları 75,1 ms–752,8 ms (ölçüm).

Tablo 5.12
Terim sözlüğü

Türkçe	İngilizce	Kısa tanım
sıralama problemi	sorting problem	öğeleri bir ölçüte göre dizmek (çıktı: sıralı permütasyon)
seçmeli sıralama	selection sort	kalanların en küçüğünü seçip öneğe takas
kabarcık sıralaması	bubble sort	komşu takasıyla büyükleri sona taşımak
araya ekleme sıralaması	insertion sort	anahtarı sıralı öneğe kaydırarak yerleştirmek
ters çevrim	inversion	i < j ve a[i] > a[j] olan çift

Türkçe	İngilizce	Kısa tanım
takas; kaydırma	swap; shift	iki öğenin yer değiştirmesi; bir öğenin bir konum kopyalanması
kararlı	stable	eşit anahtarlıların göreli sırası korunur
yerinde	in-place	O(1) ek bellek
uyarlanabilir	adaptive	sıralıya yakın girdide daha az iş
çevrimiçi	online	veri geldikçe işleyebilen
karşılaştırıcı; yüklem	comparator; predicate	iki öğenin sırasını söyleyen fonksiyon
katı zayıf sıralama	strict weak ordering	C++ yükleminin uyması gereken sıralama koşulu
sözlük sırası	lexicographic order	anahtarları önem sırasıyla karşılaştırmak
dolaylı sıralama	indirect sort	kayıtlar yerine işaretçileri/indisleri sıralamak
kâhin; karşılaştırmalı test	test oracle; differential testing	güvenilir bir başvuruyla sonuç karşılaştırma
tanımsız davranış	undefined behavior	standardın hiçbir sonuç öngörmediği durum

15. Kaynaklar

Astrachan, O. (2003). Bubble sort: An archaeological algorithmic analysis. *ACM SIGCSE Bulletin*, *35*(1), 1–5. https://doi.org/10.1145/792548.611918

Ciura, M. (2001). Best increments for the average case of Shellsort. In R. Freivalds (Ed.), *Fundamentals of computation theory: 13th International Symposium, FCT 2001* (Lecture Notes in Computer Science, Vol. 2138, pp. 106–117). Springer. https://doi.org/10.1007/3-540-44669-9_12

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

GNU Project. (t.y.). *Porting to GCC 14*. Erişim tarihi 13 Ekim 2026, https://gcc.gnu.org/gcc-14/porting_to.html

ISO/IEC. (2017). *Programming languages — C++* (ISO/IEC Standard No. 14882:2017). International Organization for Standardization.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

McLaughlin, A. (2007, November 15). *Candidates at Google: Barack Obama*. Google Public Policy Blog. https://publicpolicy.googleblog.com/2007/11/candidates-at-google-barack-obama.html

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience*, *27*(8), 983–993.

Python Software Foundation. (t.y.). *The Python standard library*. Erişim tarihi 12 Ekim 2026, https://docs.python.org/3/library/

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Tshukudu, E., & Cutts, Q. (2020). Understanding conceptual transfer for students learning new programming languages. In *Proceedings of the 2020 ACM Conference on International Computing Education Research* (pp. 227–237). ACM. https://doi.org/10.1145/3372782.3406270

Weiss, M. A. (1997). *Data structures and algorithm analysis in C* (2nd ed.). Addison-Wesley.

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on üç basamağının C++ ve Python sürümlerinden algoritmanın çekirdeğini oluşturan fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda belirtilir. Örnek 1 ve 2'nin C++ ve Python sürümlerinden Bölüm 4'te yalnızca farklılaşan satırlar anılır, tam fonksiyonları burada verilir; Örnek 3'ün `yerlestir` kesiti Bölüm 4.3'te, burada onu çağıran `araya_ekleme_sirala` vardır. On üç örneğin hepsinin çıktısı üç dilde bayt bayt aynıdır (Δ örneği yoktur).

Tablo 5.13
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H05_03_araya_ekleme)

Dil	Derleme / çalıştırma	Ek dosya ya da bağımsız değişken
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c -o X · ./X`	Örnek 4, 5, 6, 11, 13: + ortak_rastgele.c; Örnek 12: + ortak_rastgele.c zamanlayici.c
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Hepsi	`Örnek 12 isteğe bağlı n alır (varsayılan 1000)`	./X 5000 — süreler stderr'e

A.1 Örnek 1 · Seçmeli sıralama

C++ **cpp/H05_01_secmeli_siralama.cpp · satır 64–83**

```cpp
static void secmeli_sirala(std::vector<int>& a, bool yazdir)
{
    std::size_t n = a.size();
    for (std::size_t i = 0; i + 1 < n; i++) {
        std::size_t en_kucuk = i;
        for (std::size_t j = i + 1; j < n; j++) {
            karsilastirma++;
            if (a[j] < a[en_kucuk]) {
                en_kucuk = j;
            }
        }
        if (en_kucuk != i) {
            std::swap(a[i], a[en_kucuk]);
            takas_sayisi++;
        }
        if (yazdir) {
            gecis_yaz(a, i, en_kucuk);
        }
    }
}
```

Python **python/H05_01_secmeli_siralama.py · satır 46–59**

```python
def secmeli_sirala(a, yazdir):
    global karsilastirma, takas_sayisi
    n = len(a)
    for i in range(n - 1):
```

```
        en_kucuk = i
        for j in range(i + 1, n):
            karsilastirma += 1
            if a[j] < a[en_kucuk]:
                en_kucuk = j
        if en_kucuk != i:
            a[i], a[en_kucuk] = a[en_kucuk], a[i]
            takas_sayisi += 1
        if yazdir:
            gecis_yaz(a, i, en_kucuk)
```

DİL PENCERESİ · Farklılaşan satırlar

Takas C'de yardımcı `takas(int *, int *)` fonksiyonuyla, C++'ta `std::swap` ile, Python'da çoklu atamayla (`a[i], a[j] = a[j], a[i]`) yapılır. C ve C++'ta `size_t` işaretsizdir: n = 0 iken `n - 1` çok büyük bir sayıya sarar, bu yüzden döngü koşulu `i + 1 < n` yazılır; Python'da `range(n - 1)` n = 0 için boş aralıktır. (Gösterilen kesit dışındaki fark satırları: C++ 104; Python 78.)

A.2 Örnek 2 · Kabarcık sıralaması (erken çıkışlı)

C++ **cpp/H05_02_kabarcik_siralama.cpp · satır 34–60**

```
static void kabarcik_sirala(std::vector<int>& a, bool yazdir)
{
    std::size_t n = a.size();
    for (std::size_t i = 0; i + 1 < n; i++) {
        bool takas_oldu = false;          // bu geçişte takas var mı?
        long long onceki = takas_sayisi;
        for (std::size_t j = 0; j + 1 < n - i; j++) {
            karsilastirma++;
            if (a[j] > a[j + 1]) {        // komşular ters sırada
                std::swap(a[j], a[j + 1]);
                takas_sayisi++;
                takas_oldu = true;
            }
        }
        gecis_sayisi++;
        if (yazdir) {
            std::cout << "geçiş " << gecis_sayisi << ": karşılaştırma = "
                      << n - 1 - i << ", takas = "
                      << takas_sayisi - onceki << "\n";
            std::cout << "      ";
            dizi_yaz(a, n - 1 - i);
        }
        if (!takas_oldu) {
            break;                        // erken çıkış: dizi sıralı
        }
    }
}
```

Python **python/H05_02_kabarcik_siralama.py · satır 24–43**

```
def kabarcik_sirala(a, yazdir):
```

```
    global karsilastirma, takas_sayisi, gecis_sayisi
    n = len(a)
    for i in range(n - 1):
        takas_oldu = False                # bu geçişte takas var mı?
        onceki = takas_sayisi
        for j in range(n - 1 - i):
            karsilastirma += 1
            if a[j] > a[j + 1]:           # komşular ters sırada
                a[j], a[j + 1] = a[j + 1], a[j]
                takas_sayisi += 1
                takas_oldu = True
        gecis_sayisi += 1
        if yazdir:
            print(f"geçiş {gecis_sayisi}: karşılaştırma = {n - 1 - i}, "
                  f"takas = {takas_sayisi - onceki}")
            print("      ", end="")
            dizi_yaz(a, n - 1 - i)
        if not takas_oldu:
            break                         # erken çıkış: dizi sıralı
```

DİL PENCERESİ · **Farklılaşan satırlar**

Erken çıkış bayrağı C'de `int` (0/1), C++ ve Python'da `bool`'dur. C++'taki `calistir` vektörü değerle alır (kopya; `{1, 2, …}` listesinden geçici vektör kurulur), C'de dizi işaretçi olarak geçer. İç döngü sınırı C/C++'ta `j + 1 < n - i`, Python'da `range(n - 1 - i)` (üst sınır dahil değil). (Gösterilen kesit dışındaki fark satırları: C++ 62, 88.)

A.3 Örnek 3 · Araya ekleme sıralaması

C++ **cpp/H05_03_araya_ekleme.cpp · satır 51–65**

```
static void araya_ekleme_sirala(std::vector<int>& a, bool yazdir)
{
    for (std::size_t i = 1; i < a.size(); i++) {
        int anahtar = a[i];
        long long onceki = kaydirma;
        std::size_t yer = yerlestir(a, i);
        if (yazdir) {
            std::cout << "i = " << i << ": anahtar = " << std::setw(2)
                      << anahtar << ", kaydırma = " << kaydirma - onceki
                      << ", yeri a[" << yer << "]\n";
            std::cout << "      ";
            dizi_yaz(a, i + 1);
        }
    }
}
```

Python **python/H05_03_araya_ekleme.py · satır 42–51**

```
def araya_ekleme_sirala(a, yazdir):
    for i in range(1, len(a)):
        anahtar = a[i]
        onceki = kaydirma
```

```
        yer = yerlestir(a, i)
        if yazdir:
            print(f"i = {i}: anahtar = {anahtar:2d}, kaydırma = "
                  f"{kaydirma - onceki}, yeri a[{yer}]")
            print("      ", end="")
            dizi_yaz(a, i + 1)
```

DİL PENCERESİ · Farklılaşan satırlar

İç döngüde j −1'e inebildiği için C'de işaretli `ptrdiff_t` kullanılır; C++'ta `auto j = static_cast<std::ptrdiff_t>(i) - 1` aynı türü verir (`auto`: tür sağ taraftan çıkarılır). `size_t` ile `j >= 0` her zaman doğru olurdu. `&&` / `and` kısa devre yapar: `j >= 0` yanlışsa `a[j]` hiç okunmaz (Hata Avcısı 2). Çevrimiçi eklemede C sabit kapasiteli diziye `a[n] = x` yazar; C++'ta `push_back`, Python'da `append` diziyi büyütür. (Gösterilen kesit dışındaki fark satırları: C++ 41, 80, 83; Python 34, 65, 68.)

A.4 Örnek 4 · Üç algoritma, üç girdi: sayaçlar

C++ **cpp/H05_04_sayac_karsilastirma.cpp · satır 32–41**

```
32 static void sola_yasla(const std::string& s, int g)
33 {
34     int uzunluk = 0;
35     for (unsigned char c : s) {
36         if ((c & 0xC0) != 0x80) {
37             uzunluk++;
38         }
39     }
40     std::cout << s << std::string(g > uzunluk ? g - uzunluk : 0, ' ');
41 }
```

Python **python/H05_04_sayac_karsilastirma.py · satır 24–26**

```
24 def sola_yasla(s, g):
25     """s'yi yazar; görünen genişlik (kod noktası) g olana dek boşluk"""
26     print(f"{s:<{g}}", end="")
```

DİL PENCERESİ · Farklılaşan satırlar

Türkçe etiketleri hizalamak için C ve C++'ta UTF-8 baytlarından kod noktası sayılır (`(c & 0xC0) != 0x80`): `printf("%-14s")` ve `std::setw` BAYT sayar, Python'un `f"{s:<14}"` biçimi ise KARAKTER sayar. Karıştırma C'de `rastgele_karistir(a, n)`, C++'ta `rastgele_karistir(a)` (vektör), Python'da `rastgele_karistir(a)` (liste) ile yapılır; üç dilde aynı permütasyon çıkar. (Gösterilen kesit dışındaki fark satırları: C++ 139; Python 99, 102.)

A.5 Örnek 5 · Ters çevrim ve kaydırma

C++ **cpp/H05_05_ters_cevrim.cpp · satır 21–32**

```
21 static long long ters_cevrim_say(const std::vector<int>& a)
22 {
```

```cpp
    long long sayi = 0;
    for (std::size_t i = 0; i < a.size(); i++) {
        for (std::size_t j = i + 1; j < a.size(); j++) {
            if (a[i] > a[j]) {
                sayi++;
            }
        }
    }
    return sayi;
}
```

Python `python/H05_05_ters_cevrim.py · satır 14–21`

```python
def ters_cevrim_say(a):
    """Ters çevrim: i < j ve a[i] > a[j] olan (i, j) çiftlerinin sayısı"""
    sayi = 0
    for i in range(len(a)):
        for j in range(i + 1, len(a)):
            if a[i] > a[j]:
                sayi += 1
    return sayi
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de `rastgele_dizi(a, m, 0, 999)` var olan diziyi doldurur; C++ ve Python'da yeni vektör/liste döndürür. C++'ta `b = a` vektörü kopyalar, Python'da kopya için `a[:]` gerekir. Ters çevrim sayısı n(n−1)/2'ye ulaşabildiği için C/C++'ta sayaç `long long`'dur; Python tamsayısı taşmaz. (Gösterilen kesit dışındaki fark satırları: C++ 84, 106, 109; Python 64, 83, 89.)

A.6 Örnek 6 · "Sıralı mı?" ve kâhin testi

C++ `cpp/H05_06_kahin_testi.cpp · satır 81–96`

```cpp
static bool sirali_mi(const std::vector<int>& a)
{
    for (std::size_t k = 1; k < a.size(); k++) {
        if (a[k - 1] > a[k]) {
            return false;
        }
    }
    return true;
}

// Kâhin: kütüphane sıralamasının sonucu (yalnızca test için)
static std::vector<int> kahin(std::vector<int> girdi)
{
    std::sort(girdi.begin(), girdi.end());
    return girdi;
}
```

Python `python/H05_06_kahin_testi.py · satır 56–66`

```python
def sirali_mi(a):
    """a azalmayan sırada mı? (n - 1 karşılaştırma)"""
    for k in range(1, len(a)):
        if a[k - 1] > a[k]:
            return False
    return True


def kahin(girdi):
    """Kâhin: kütüphane sıralamasının sonucu (yalnızca test için)"""
    return sorted(girdi)
```

DİL PENCERESİ · **Farklılaşan satırlar**

Algoritmalar C'de fonksiyon işaretçisi dizisinde (`typedef void (*siralayici)(int *, size_t)`), C++'ta `using` ile adlandırılmış işaretçi türünde, Python'da düz bir listede tutulur (fonksiyonlar birinci sınıf değerdir). Dizi eşitliği C'de `memcmp`, C++'ta vektör `==`, Python'da liste `==` ile denetlenir; kâhin C'de `qsort`, C++'ta `std::sort`, Python'da `sorted`. (Gösterilen kesit dışındaki fark satırları: C++ 21, 161; Python 92, 111.)

A.7 Örnek 7 · qsort ve taşmaya güvenli karşılaştırıcı

C++ **cpp/H05_07_qsort_arayuzu.cpp · satır 65–79**

```cpp
    std::sort(a.begin(), a.end());                       // x < y
    int_yaz("artan ", a);
    std::sort(a.begin(), a.end(), std::greater<int>());  // x > y
    int_yaz("azalan", a);
    long long fark = static_cast<long long>(INT_MAX) - (-1);
    std::cout << "   neden x - y değil? INT_MAX - (-1) = " << fark
              << " > INT_MAX = " << INT_MAX << "\n";
    std::cout << "   (int'te taşma: C'de tanımsız davranış; bkz. Hata "
                 "Avcısı)\n";

    std::vector<double> d = {2.5, -0.75, 3.14, 0.1, -2.0, 0.3};
    std::cout << "\n2) double dizisi\n";
    double_yaz("girdi ", d);
    std::sort(d.begin(), d.end(),
              [](double x, double y) { return x < y; });  // lambda
```

Python **python/H05_07_qsort_arayuzu.py · satır 36–64**

```python
    a = sorted(a, key=functools.cmp_to_key(int_artan))
    int_yaz("artan ", a)
    a = sorted(a, reverse=True)
    int_yaz("azalan", a)
    fark = INT_MAX - (-1)
    print(f"   neden x - y değil? INT_MAX - (-1) = {fark} > INT_MAX = "
          f"{INT_MAX}")
    print("   (int'te taşma: C'de tanımsız davranış; bkz. Hata Avcısı)")

    d = [2.5, -0.75, 3.14, 0.1, -2.0, 0.3]
    print("\n2) double dizisi")
    double_yaz("girdi ", d)
```

```
    d = sorted(d)
    double_yaz("artan ", d)
    print(f"   (int)(0.1 - 0.3) = {int(0.1 - 0.3)}: kesme, farklı "
          "sayıları \"eşit\" gösterir")

    s = ["elma", "Zeki", "armut", "Çelik", "Can", "ali", "Şule"]
    print("\n3) dizgi dizisi: strcmp (bayt sırası)")
    dizgi_yaz("girdi ", s)
    s.sort()                # yerinde; kod noktası = UTF-8 bayt sırası
    dizgi_yaz("artan ", s)
    print("   bayt sırası alfabe sırası değil: Ç ve Ş, z'den bile sonra "
          "(Mini-OBS 2 v2.4)")

    m = ["elma", "kivi", "armut", "nar", "incir", "ayva", "dut"]
    print("\n4) iki anahtarlı karşılaştırıcı: uzunluk, sonra alfabe")
    dizgi_yaz("girdi ", m)
    m = sorted(m, key=lambda x: (len(x), x))   # demet: sözlük sırası
```

DİL PENCERESİ · Farklılaşan satırlar

C'de karşılaştırıcı öğelerin ADRESLERİNİ `const void *` olarak alır ve int (< 0, 0, > 0) döndürür; C++'ta `bool` döndüren "küçüktür" yüklemi (lambda ya da `std::greater<int>()`), Python'da `key=` anahtar fonksiyonu, `reverse=True` ya da `functools.cmp_to_key` ile C tarzı karşılaştırıcı kullanılır. Python tamsayısı sınırsız olduğundan `x - y` orada taşmaz; C'de int taşması tanımsız davranıştır. `s.sort()` yerinde sıralar ve `None` döndürür, `sorted()` yeni liste üretir. (Gösterilen kesit dışındaki fark satırları: C++ 23.)

A.8 Örnek 8 · Yapı dizisini sıralama

C++ **cpp/H05_08_yapi_siralama.cpp · satır 26–42**

```
template <class Once>
static void ogrenci_sirala(std::vector<Ogrenci>& a, Once once_mi)
{
    for (std::size_t i = 1; i < a.size(); i++) {
        Ogrenci anahtar = a[i];             // yapı ataması: kopya
        auto j = static_cast<std::ptrdiff_t>(i) - 1;
        while (j >= 0) {
            karsilastirma++;
            if (!once_mi(anahtar, a[j])) {
                break;
            }
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = anahtar;
    }
}
```

Python **python/H05_08_yapi_siralama.py · satır 16–28**

```
def ogrenci_sirala(a, anahtar_fn):
    """Araya ekleme: anahtar_fn(a[j]) > anahtar_fn(x) oldukça kaydır"""
    global karsilastirma
```

```
    for i in range(1, len(a)):
        anahtar = a[i]                      # başvuru (kopya değil)
        j = i - 1
        while j >= 0:
            karsilastirma += 1
            if anahtar_fn(a[j]) <= anahtar_fn(anahtar):
                break
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = anahtar
```

DİL PENCERESİ · **Farklılaşan satırlar**

C karşılaştırıcısı üç değerli `int` döndürür (azalan sıra için işaretler ters çevrilir); C++'ta "x, y'den önce mi?" yüklemi lambda ile verilir ve şablon (template) parametresiyle alınır; Python'da anahtar fonksiyonu `-o.ortalama` döndürür. C ve C++'ta `Ogrenci anahtar = a[i]` yapının tamamını kopyalar; Python'da aynı yazım yalnızca nesne başvurusunu kopyalar. (Gösterilen kesit dışındaki fark satırları: C++ 78; Python 11, 61.)

A.9 Örnek 9 · Kararlılık deneyi

C++ **cpp/H05_09_kararlilik.cpp · satır 65–76**

```
static void araya_ekleme_sirala(std::vector<Ogrenci>& a, OnceMi once_mi)
{
    for (std::size_t i = 1; i < a.size(); i++) {
        Ogrenci anahtar = a[i];
        auto j = static_cast<std::ptrdiff_t>(i) - 1;
        while (j >= 0 && once_mi(anahtar, a[j])) {   // eşitse dur
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = anahtar;
    }
}
```

Python **python/H05_09_kararlilik.py · satır 42–49**

```
def araya_ekleme_sirala(a, anahtar_fn):
    for i in range(1, len(a)):
        anahtar = a[i]
        j = i - 1
        while j >= 0 and anahtar_fn(a[j]) > anahtar_fn(anahtar):  # eşit: dur
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = anahtar
```

DİL PENCERESİ · **Farklılaşan satırlar**

Algoritma ve karşılaştırıcı C'de fonksiyon işaretçisiyle, C++'ta `using` ile adlandırılmış işaretçi türleriyle, Python'da doğrudan fonksiyon değeriyle aktarılır. Kopya C'de `memcpy`, C++'ta vektör ataması, Python'da `girdi[:]` dilimiyle alınır (sığ kopya; `namedtuple` değiştirilemez olduğundan yeterlidir). (Gösterilen kesit dışındaki fark satırları: C++ 24, 43, 138; Python 16, 27, 102.)

A.10 Örnek 10 · Çok anahtarlı sıralama

C++ cpp/H05_10_cok_anahtarli.cpp · satır 44–53

```cpp
static bool cok_anahtarli(const Ogrenci& x, const Ogrenci& y)
{
    if (x.ortalama != y.ortalama) {
        return x.ortalama > y.ortalama;
    }
    if (x.soyad != y.soyad) {
        return x.soyad < y.soyad;
    }
    return x.ad < y.ad;
}
```

Python python/H05_10_cok_anahtarli.py · satır 28–30

```python
def cok_anahtarli(o):
    """Tek anahtar: demet (tuple) sözlük sırasıyla karşılaştırılır"""
    return (-o.ortalama, o.soyad, o.ad)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Python'da çok anahtarlı sıra demetle (tuple) kurulur: `(-o.ortalama, o.soyad, o.ad)` sözlük sırasıyla karşılaştırılır. C'de anahtarlar sırayla denenir (`k != 0` ise döndür), C++'ta `std::string` için `!=` ve `<` işleçleri kullanılır. `strcmp`, C++ `std::string` ve Python `str` karşılaştırmaları UTF-8 metinde aynı sırayı verir (bayt sırası = kod noktası sırası), ama bu Türkçe alfabe sırası değildir. (Gösterilen kesit dışındaki fark satırları: C++ 35; Python 62.)

A.11 Örnek 11 · k-sıralı dizide araya ekleme

C++ cpp/H05_11_k_sirali.cpp · satır 52–63

```cpp
static std::size_t en_buyuk_uzaklik(const std::vector<int>& a)
{
    std::size_t en = 0;
    for (std::size_t i = 0; i < a.size(); i++) {
        std::size_t hedef = static_cast<std::size_t>(a[i]) - 1;
        std::size_t d = i > hedef ? i - hedef : hedef - i;
        if (d > en) {
            en = d;
        }
    }
    return en;
}
```

Python python/H05_11_k_sirali.py · satır 37–44

```python
def en_buyuk_uzaklik(a):
    """En büyük |i - sıralı konum|: değer v'nin sıralı konumu v - 1"""
    en = 0
```

```
40      for i in range(len(a)):
41          d = abs(i - (a[i] - 1))
42          if d > en:
43              en = d
44      return en
```

DİL PENCERESİ · Farklılaşan satırlar

C ve C++'ta `size_t` işaretsiz olduğundan uzaklık `i > hedef ? i - hedef : hedef - i` ile hesaplanır (`i - hedef` negatife düşmez, sarar); Python'da `abs(i - (a[i] - 1))` yeter. Azalan döngü Python'da `range(son, bas, -1)` ile yazılır. Blok karıştırma üç dilde aynı `rastgele_aralik` çağrı sırasını kullanır; diziler birebir aynıdır. (Gösterilen kesit dışındaki fark satırları: C++ 137, 163; Python 29, 31.)

A.12 Örnek 12 · Dolaylı sıralama

C++ **cpp/H05_12_dolayli_siralama.cpp · satır 51–65**

```
51  static void isaretci_sirala(std::vector<const Kayit*>& p)
52  {
53      for (std::size_t i = 1; i < p.size(); i++) {
54          const Kayit* anahtar = p[i];
55          kopya++;
56          auto j = static_cast<std::ptrdiff_t>(i) - 1;
57          while (j >= 0 && p[j]->anahtar > anahtar->anahtar) {
58              p[j + 1] = p[j];
59              kopya++;
60              j--;
61          }
62          p[j + 1] = anahtar;
63          kopya++;
64      }
65  }
```

Python **python/H05_12_dolayli_siralama.py · satır 38–50**

```
38  def indis_sirala(sira, kayit):
39      """Dolaylı: indisler kaydırılır; kayıt listesi yerinde kalır"""
40      global kopya
41      for i in range(1, len(sira)):
42          anahtar = sira[i]
43          kopya += 1
44          j = i - 1
45          while j >= 0 and kayit[sira[j]].anahtar > kayit[anahtar].anahtar:
46              sira[j + 1] = sira[j]
47              kopya += 1
48              j -= 1
49          sira[j + 1] = anahtar
50          kopya += 1
```

DİL PENCERESİ · Farklılaşan satırlar

C ve C++'ta dizi yapıların kendisini tutar: her kaydırma 256 baytlık bir kopyadır; işaretçi dizisi aynı sırayı 8

baytlık kopyalarla verir. Python listesi her zaman nesne başvurusu tuttuğundan oradaki "doğrudan" sıralama da aslında başvuruları taşır; bayt sayıları C karşılığını göstermek için sabitlerle (`KAYIT_BAYT`, `ISARETCI_BAYT`) hesaplanır. Süreler stderr'e yazılır. (Gösterilen kesit dışındaki fark satırları: C++ 26, 29, 98; Python 17, 18, 71.)

A.13 Örnek 13 · Shell sıralaması (merak)

C++ **cpp/H05_13_shell_siralama.cpp · satır 31–47**

```cpp
static void h_gecisi(std::vector<int>& a, std::size_t h)
{
    for (std::size_t i = h; i < a.size(); i++) {
        int anahtar = a[i];
        std::size_t j = i;                 // boşluğun konumu
        while (j >= h) {
            karsilastirma++;
            if (a[j - h] <= anahtar) {
                break;
            }
            a[j] = a[j - h];   // h sağa kaydır
            kaydirma++;
            j -= h;
        }
        a[j] = anahtar;
    }
}
```

Python **python/H05_13_shell_siralama.py · satır 20–33**

```python
def h_gecisi(a, h):
    """Her h için: a[i], a[i-h], a[i-2h], ... alt dizisinde araya ekleme"""
    global karsilastirma, kaydirma
    for i in range(h, len(a)):
        anahtar = a[i]
        j = i                                # boşluğun konumu
        while j >= h:
            karsilastirma += 1
            if a[j - h] <= anahtar:
                break
            a[j] = a[j - h]    # h sağa kaydır
            kaydirma += 1
            j -= h
        a[j] = anahtar
```

DİL PENCERESİ · **Farklılaşan satırlar**

Aralık dizisi C'de sabit dizi + eleman sayısıyla (`size_t h[MAKS_H]`), C++'ta `std::vector<std::size_t>`, Python'da liste olarak döndürülür. `n / 2` C/C++'ta tamsayı bölmesidir; Python'da `n // 2` yazılır (`/` ondalık sonuç verir). Boşluk indisi `j >= h` koşuluyla korunduğundan işaretsiz `size_t` ile `j - h` negatife düşmez. (Gösterilen kesit dışındaki fark satırları: C++ 52, 69, 153; Python 55, 61, 72.)